

ChatGPT - LRH Project

This is super interesting. Let's talk about tiers of options.

One of the sets of options you presented went from "Option A: Just link out to other tools" through "Option F: Full LRH Agentic Chat" with B-C being the near-term recommended defaults. If we had a billion dollars, we might do Option F, but, conversely, one of the attractions of "Option A: Link to Other Tools" from a user-centric, user-friendly perspective is that if tomorrow ChatGPT, Codex or NovumAI (wow, I just made that up and there's already a company called that) makes a new great tool, users should be able to use it in the LRH flow.

Another set of options you presented is how the chats are stored from "Option 1: Store in the Repo" to "Option 4: A private control repo". Interestingly, we've already started to think of Option 4 on the grounds that some projects may want to separate the control plane from the public repo, or to have one control plane coordinate multiple repos. As a simple example, let's say we want to release the Replication Vector game on Steam. This might involve a purely public repo for the Velumin graphics library, a second probably public repo for the Replication Vector game engine, a possibly private repo for Replication Vector's game assets and levels, and a definitely private repo containing the Steam client publishing information. One control plane might be the solution for managing all of these.

The overall summary is that your proposed design of Lrh conversaitons seems good, but:

- we know we may want to separate out repos and should consider that as a tool. E.g., private state might be a local git repository. I am fully aware that using git as your database presents significant hazards that I am not fully up to date on, so thinking through how private state can be stored in a safe, preserved way is very important here as we work through the design. A persistent, stateful, logged backing store for conversations would be very important
- since we know tomorrow a new agent tool may be developed that might have a great persistent, stateful logged backing store with a great user interface and superior client performance on one or more tasks in the LRH computation graph (e.g., a super awesome code review tool or graphical user interface development tool), what if we flip the script and define an API where an external tool can plug into LRH and work within its framework? That is, if LRH is valuable, what if there was an API, agent skill, or something like MCP or Github Agents that would enable an external tool to somehow plug itself into the LRH framework and get used by it (or vice versa). For example, in Github one can tag codex in a chat and it will do a review for you. What if you could be chatting with ChatGPT or Codex and kick off an LRH run? I don't know how this would work, but I want to raise the issue of - in addition to the design you proposed - establishing a clear protocol that other agents could plug into could be very valuable and it seems worth thinking through.

What are the options for doing this?

What are their pros and cons with respect to best practices?
Does any one of them have a decisive advantage or a disqualifying limitation?

Let's think step by step and check our work to make sure we get

the right answer, and cite or quote reputable sources in our answer so we know it is grounded.

This is the right reframing. I would separate the problem into **two independent axes**:

1. **Where does conversation / execution state live?**
2. **How do outside tools plug into LRH's workflow graph?**

Those axes should be designed together, but not conflated. Storage is about durability, privacy, audit, and recovery. Interop is about letting tomorrow's better agent/tool participate without LRH needing to clone it.

My recommended north star is:

LRH should become a transparent control-plane protocol and ledger around many tools, not a replacement for every tool.

That means LRH Conversations should not be "the one chat UI to rule them all." They should be the **auditable connective tissue** between project state, conversations, prompts, external agents, runs, evidence, and status.

1. The core principle: separate authority, storage, and execution

LRH already has the right philosophy for this. Its docs describe it as a repository-centered control plane for "designing, executing, and evaluating structured, evidence-backed, agent-assisted work," with the project's local **project/** directory authoritative even when a workspace/dashboard coordinates across projects.

humans

For chat and agent interoperability, that suggests three explicit layers:

Authoritative control artifacts

project/design, project/work_items, project/evidence, project/status

Durable conversation/runtime ledger

chats, prompts, tool invocations, run attempts, imported transcripts

External execution tools

ChatGPT, Codex, Claude Code, Copilot, Antigravity, Aider, OpenHands, future tools

Do not force these into one store or one app.

The key invariant should be:

External tools may propose, execute, report, and attach evidence, but LRH remains the place where scope, provenance, guardrails, and status are made explicit.

2. Storage tier options

Storage Option 1 — Controlled repo only

All chats and conversation artifacts live in the same public or project repo:

```
controlled-repo/
  project/
    conversations/
      runs/
        evidence/
```

Pros

Maximum visibility. Simple mental model. Everything is versioned. This aligns with Git's basic purpose: version control records changes to files over time so specific versions can be recalled later. [Git](#)

Cons

This is too risky as a default. Chats can contain secrets, client context, vendor logs, API responses, private reasoning, proprietary design notes, and accidental credential leaks. It also pollutes public repos with bulky transcripts.

Verdict

Good for curated summaries and public evidence. Bad for raw conversation state.

Disqualifying as default: raw chats in public controlled repos.

Storage Option 2 — Local private state store

Raw conversations and execution logs live in local private state, for example:

```
~/.config/lrh/private/chats/
~/.local/state/lrh/chats/
```

or an LRH workspace-private path.

Pros

Simple, private by default, and consistent with LRH's emerging split between shareable control state and private runtime state. LRH's meta workspace design already distinguishes durable shareable catalog state from global/XDG private/runtime state.

humans

Cons

Local-only storage is fragile unless backed up. It can become hard to share with collaborators. It may not be enough for team workflows.

Verdict

Best initial default for raw transcripts.

Storage Option 3 — SQLite-backed local ledger

Use a real local database for conversations, turns, attachments, tool calls, run events, and sync state.

`lrh_private_state.sqlite`

Pros

This is the strongest near-term persistence layer. SQLite is embedded, local, portable, widely understood, transactional, and durable. Its documentation emphasizes atomic commit: either all database changes in a transaction occur or none do. [SQLite](#) That matters for chat/run logs because partially-written state is a real hazard.

A database also supports:

conversation search
message IDs
event streams
attachments
sync status
redaction state
run linkage
import provenance

Cons

A database is less directly human-readable than Markdown. It needs export/import and backup behavior. It also introduces schema migration concerns.

Verdict

Best default for raw durable private state.

The right pattern is:

SQLite for durable private runtime ledger
Markdown/JSON exports for reviewed control-plane artifacts
Git for curated, versioned project state

Storage Option 4 — Git-backed private control repo

Use a separate private repository as the control plane or extended private state for one or more controlled repos.

Example for Replication Vector:

<code>replication-vector-control/</code>	<code>private LRH control repo</code>
<code>velumin/</code>	<code>public graphics library</code>
<code>replication-vector-engine/</code>	<code>public or private game engine</code>
<code>replication-vector-assets/</code>	<code>private assets/levels</code>
<code>steam-publishing/</code>	<code>private publishing information</code>

Pros

This is strategically important. It supports multi-repo products, private planning, public code, private assets, and sensitive release information. It also keeps LRH's "repo as control plane" model while acknowledging real product boundaries.

Git is distributed and gives every clone a full copy of project history, which is useful for audit and recovery. [GitHub Docs](#)

Cons

Git is not a database. It handles versioned files well, but it is not ideal for high-frequency chat append logs, concurrent writers, large binary attachments, or sensitive redaction mistakes. Rewriting history after a secret leak is painful and not always sufficient.

Verdict

Excellent for curated, reviewed, shareable control state. Risky as the only backing store for raw high-volume chat/runtime state.

The pattern should be:

SQLite/local ledger → curated export → private control repo commit

not:

every chat turn → git commit

Storage Option 5 — Server-backed LRH workspace

Run an LRH service with database-backed state for teams.

Pros

Best for collaboration, multi-user sync, authorization, backups, search, retention, and audit.

Cons

Much bigger operational burden. Now LRH needs authentication, authorization, database migrations, backups, deployment, tenancy, secrets management, and support.

Verdict

Long-term team/enterprise tier, not near-term default.

3. Storage recommendation

Use a **tiered state model**:

Tier 0: Ephemeral session
transient chat buffer, not saved

Tier 1: Private durable local ledger
SQLite by default, raw transcripts and run events

Tier 2: Curated project artifacts
Markdown/JSON committed to project/control repo

Tier 3: Private control repo
shared private control plane for one or more controlled repos

Tier 4: Team LRH service
server-backed collaboration, later

This avoids the two bad extremes:

Bad extreme A: every private chat goes into the public repo
Bad extreme B: all process remains hidden inside vendor apps

4. External-tool plug-in options

Now flip the script, as you suggest. Instead of asking “how can LRH become the best chat app?”, ask:

“How can LRH expose a stable control-plane protocol that external tools can use?”

That is the more durable idea.

Interop Option A — Human copy/paste and links

External tools remain external. LRH records links, summaries, and manually imported transcripts.

Pros

Works immediately. No API dependencies. Compatible with any future tool.

Cons

Manual, lossy, inconsistent.

Verdict

Must exist as baseline fallback.

Interop Option B — CLI-first protocol

External tools can call LRH commands:

Bash

```
lrh context get --project . --work-item WI-123 --format json
lrh run propose --from-file prompt.md --tool codex
lrh run record-event --run RUN-123 --event started
lrh evidence add --run RUN-123 --file report.md
lrh status interpret
```

Pros

Extremely practical. Works with shell scripts, Aider, Claude Code, Codex environments, CI, GitHub Actions, and future tools. Easy to test. Fits LRH's existing CLI orientation.

Cons

CLI protocols can become ad hoc unless JSON schemas are stable. Long-running bidirectional interaction is awkward.

Verdict

Best first interop layer.

Interop Option C — File-based protocol / inbox-outbox

LRH defines directories where tools can read context packets and write results:

```
.lrh_exchange/
  inbox/
    RUN-123.request.json
  outbox/
    RUN-123.result.json
  attachments/
```

or under private state.

Pros

Simple, debuggable, offline-friendly, language-agnostic. Great for tools that cannot easily call a server.

Cons

Needs locking, atomic writes, schema validation, and cleanup rules. Concurrent writers are tricky.

Verdict

Good companion to CLI, especially for robust offline workflows.

Use atomic file write discipline or SQLite-backed queue semantics; do not rely on casual partial writes.

Interop Option D — LRH MCP server

LRH exposes its control plane through Model Context Protocol.

This is attractive because MCP is explicitly designed as an open protocol connecting LLM applications with external data sources and tools. The MCP spec describes it as a standardized way to connect LLMs with needed context, and its resource model lets servers expose application-specific information by URI.

[Model Context Protocol+1](#)

LRH could expose MCP resources:

```
lrh://projects
lrh://project/LRH/snapshot
lrh://project/LRH/work-items/WI-123
lrh://project/LRH/conversations/CHAT-456
```

and MCP tools:

```
lrh.validate
lrh.snapshot
lrh.chat.capture
lrh.run.propose
lrh.evidence.attach
lrh.status.interpret
```

Pros

This is the natural “external AI tool plugs into LRH” route. If ChatGPT, Claude Desktop, Cursor, Codex-like tools, or future systems support MCP-style integration, LRH can be a context/tool server rather than the UI.

MCP also includes authorization capabilities at the transport level for HTTP-based transports, which is important for restricted LRH resources. [Model Context Protocol](#)

Cons

MCP is still an evolving ecosystem. Client behavior varies. Authentication, local server exposure, and tool permissions need careful design. MCP should not be the only protocol.

Verdict

Decisive mid-term advantage for external AI tool interoperability.

Not first-only, but likely the most important protocol surface.

Interop Option E — GitHub App / PR-comment integration

LRH integrates into GitHub: comments, PR review commands, checks, labels, or slash commands.

Example:

```
/lrh snapshot  
/lrh run propose WI-123  
/lrh attach-evidence  
/lrh ask status
```

GitHub Apps are a good fit for this because GitHub says app permissions determine what APIs and webhooks the app can access, and recommends choosing minimum required permissions. [GitHub Docs](#)
GitHub also notes that webhooks let apps receive real-time notifications for events such as pushes and PRs. [GitHub Docs](#)

Pros

Excellent for PR-driven workflows. Natural place to connect Codex/Copilot review loops with LRH.

Cons

GitHub-specific. Requires app setup, permissions, webhook security, hosting, and possibly public internet exposure. Does not help local-only workflows.

Verdict

Great later adapter. Not the core protocol.

Interop Option F — Native LRH plugin/agent SDK

Define a Python API and plugin interface:

```
Python  
class LRHToolAdapter:  
    def capabilities(self) -> ToolCapabilities: ...
```



```
def propose_run(self, request: RunRequest) -> RunProposal: ...  
def execute(self, run: ApprovedRun) -> RunResult: ...
```

Pros

Good for LRH-native tools and local extensions.

Cons

Python-only unless wrapped. More maintenance. Plugins can become security risks if loaded dynamically.

Verdict

Useful, but do not make arbitrary plugins the first security boundary.

Interop Option G — Hosted LRH API

Run a local or remote HTTP service:

```
GET /projects  
GET /projects/{id}/snapshot  
POST /runs/proposals  
POST /runs/{id}/events  
POST /evidence
```

Pros

Language-agnostic, easy for GUIs and external tools.

Cons

Authentication, authorization, local exposure, CSRF/CORS, secrets, and network security become immediate concerns.

Verdict

Good after local CLI/file/MCP semantics are stable. For `lrh serve`, start local-only and read-only unless explicitly configured.

5. Protocol design: what should external tools actually do?

I would define an **LRH Interop Protocol** around six primitives.

1. Context

External tool asks:

What is the project/workstream/work-item context?

LRH returns a scoped context packet:

JSON

```
{
  "project": "...",
  "scope": {"type": "work_item", "id": "WI-123"},
  "snapshot": "...",
  "constraints": [],
  "evidence": [],
  "allowed_actions": []
}
```

2. Proposal

External tool says:

Here is what I propose to do.

LRH records a run proposal, not an execution.

3. Decision

Human or policy layer says:

Approved / rejected / needs revision.

4. Execution events

External tool reports:

started
progress
blocked
completed
failed
cancelled

5. Evidence

External tool attaches:

diffs
logs
test output
screenshots
PR links
review notes
artifacts

6. Status interpretation

LRH interprets:

What does this evidence mean for the work item/focus/project?

This maps exactly to LRH's current workflow: work is executed, evidence is collected, and status is derived from evidence and validation, not merely asserted.

humans

6. The “tag LRH from another tool” idea

You asked: what if you could be chatting with ChatGPT or Codex and kick off an LRH run?

There are several possible mechanisms.

Mechanism 1 — Copy/paste command block

External chat produces:

Bash

```
lh run propose --work-item WI-123 --prompt-file prompt.md --tool codex
```

Simple and robust.

Mechanism 2 — MCP tool call

The external client calls:

```
lh.run.propose(...)
```

This is elegant if the client supports MCP.

Mechanism 3 — GitHub comment command

In PR or issue:

```
/lh run propose WI-123 using codex
```

A GitHub App receives webhook, validates permission, and creates a proposal.

Mechanism 4 — Local URL/deep link

External app opens:

```
lh://run/propose?work_item=WI-123
```

or a localhost URL.

Useful for desktop UI; risky unless carefully permissioned.

Mechanism 5 — Inbox file

External tool writes:

```
~/.local/state/lh/inbox/run-proposal-123.json
```

LRH validates and imports it.

Very robust for offline tools.

7. Best-practice security boundary

The major rule should be:

External tools can submit proposals and evidence. They cannot silently mutate authoritative state or start dangerous runs.

This matters because prompt injection is real. OWASP describes prompt injection as malicious input that changes intended model behavior, and notes that it can lead to unauthorized actions or sensitive

[OWASP Cheat Sheet](#)

information disclosure. [Series+1](#)

Therefore:

```
external tool says: "I propose..."
LRH says: "This is in/out of scope..."
human or policy says: "Approved..."
only then: execution happens
```

This preserves LRH's control-plane philosophy.

8. Decisive recommendation

The decisive architecture is a **hub-and-spoke LRH protocol**:

```
LRH = control-plane hub
External tools = specialized spokes
```

With storage:

```
SQLite/local ledger for raw private runtime state
Markdown/JSON in repo for curated control artifacts
Private control repo for multi-repo products
MCP/CLI/file protocol for external tools
GitHub App later for PR-native workflows
```

This has the best long-term fit because it lets LRH benefit from every future "NovumAI" without surrendering its core value: auditability, explicit state, guardrails, and evidence.

9. Disqualifying limitations

Disqualify: Git as the only chat database

Git is excellent for versioning curated artifacts, but raw chat/runtime state has different needs: high-frequency append, search, redaction, attachments, partial writes, local privacy, and concurrent access. Use Git for reviewed exports and control state, not as the only live database.

Disqualify: public raw transcript storage by default

Too much privacy and secrets risk.

Disqualify: direct external-agent mutation of LRH state

External tools must not directly mark work complete, update status, or launch privileged actions without LRH validation/guardrails.

Disqualify: vendor scraping as core architecture

Scraping ChatGPT/Codex/Claude app UIs or opaque histories should be optional best-effort import, never the foundation.

Disqualify: one protocol only

MCP is promising, but not every tool supports it. CLI and file-based contracts are still necessary.

10. Recommended staged plan

Phase 0 — Design proposal

Add a design proposal for:

LRH Conversations, Private State, and External Agent Interop

It should cover:

- storage tiers
- conversation schema
- private control repo pattern
- external protocol primitives
- MCP/CLI/file/GitHub adapter surfaces
- security and permission model
- promotion path from chat → run → evidence → status

Phase 1 — Storage foundation

Implement:

- private local SQLite ledger
- conversation/event schema
- export/import to Markdown/JSON
- redaction markers
- backup/export command

Phase 2 — Manual and file-based interop

Implement:

```
lrh conversation import
lrh conversation export
lrh run proposal import/export
```

Phase 3 — CLI protocol

Implement stable JSON commands:

```
lrh context get --format json
lrh run propose --format json
lrh run event append --format json
lrh evidence attach --format json
```

Phase 4 — MCP server

Expose LRH context/resources/tools to MCP-capable clients.

Phase 5 — GitHub App / PR adapter

Add PR comment commands, status checks, webhook import, and evidence attachment.

Phase 6 — Native chat UI

Only then build a richer LRH chat panel that uses the same protocol.

Final answer

The strongest option is not “LRH builds the best chat app.” It is:

LRH defines the durable state model and interop protocol that lets any chat or agent tool participate in a transparent, evidence-backed workflow.

The decisive design choice is:

Private durable ledger + curated repo artifacts + external agent protocol

That lets LRH support:

- user choice of tools
- future best-of-breed agents
- private multi-repo control planes
- transparent runs
- auditable conversations
- evidence-backed status
- safe human approval boundaries

while avoiding the traps of public raw transcripts, Git-as-live-database, vendor lock-in, and premature full-agentic chat.

This sounds really good. I like this framing. One area I think may need a bit of steelmanning, and it suggests a second.

- Storage foundation: it seems to me that this is a really good summary of the technical scope and risks. I wonder, however, if it would be worth going a notch further and defining an LRH storage API (perhaps this is what you meant, but just to unpack it) which defines (a) a clear set of operations with (b) durability guarantees and versioning guarantees unless things are marked transient and (c) privacy guarantees unless things are marked public. So, for example, imagine that chats are stored in a conceptual space of `/transient/<group>/<session_id>/<turn>` which lives locally until say reboot and `/versioned/private/<group>/<session_id>/<turn>` and `/versioned/public/...` and so on which are guaranteed to be versioned and live but with different publicity, with a clear set of operations for each chat at each endpoint. I am not proposing this as a design per se but as a talking point to think through how to make this work. For example "In massive internal monorepo environments like Google's, storing retention metadata for billions of temporary files creates massive database overhead. To bypass this, internal systems use convention-over-configuration, encoding the retention metadata directly into the directory namespace string itself (e.g. `/ttl=30d/`). Decoupled background daemons then parse these paths to execute garbage collection, entirely removing the need for a separate policy metadata lookup." Something like that might work for this use case, though Google eventually moved to separate metadata. If we work this conceptual foundation out, the storage layer's development tech tree might become very clear.

- Conceptual Conversation API: Assume the storage foundation is well defined. You mentioned a CLI protocol and later MCP integration. But I wonder if steelmanning the conversation interface you proposed first, augmented with the ideas we discussed later, might be worthwhile. Again, if we have a solid conceptual foundation here, building out the rest may be very clear.

This would fold back into Phase 0 by putting a little more upfront effort in a few of the first items on your list:

- storage tiers
- conversation schema
- private control repo pattern
- external protocol primitives
- MCP/CLI/file/GitHub adapter surfaces
- security and permission model
- promotion path from chat → run → evidence → status

becomes a bit more elaborate:

- storage model
 - security / privacy model
 - transience / durability model
 - storage tiers (conceptual model)
 - private repo control
- conversation model
 - conversation schema
 - conversation API

- external protocol primitives
- external adapter surfaces (not implementation)

What are the options for doing this?

What are their pros and cons with respect to best practices?

Does any one of them have a decisive advantage or a disqualifying limitation?

Let's think step by step and check our work to make sure we get the right answer, and cite or quote reputable sources in our answer so we know it is grounded.

Yes. I think your refinement is exactly the right next level of abstraction.

The earlier framing was:

private durable ledger + curated repo artifacts + external agent protocol

Your refinement says: do not jump straight from that phrase to "SQLite plus a few commands." First define:

LRH Storage API
LRH Conversation API

That is the stronger architecture. It lets LRH avoid binding itself too early to SQLite, Git, MCP, a web UI, or a specific vendor agent.

The key recommendation is:

Define **conceptual APIs first**, then implement the smallest local backend and CLI surface that satisfies them.

That fits LRH's own engineering discipline: the repository guidance says to preserve clear boundaries between harness code, tests, project control plane, future client project repositories, and runtime structured objects; it also warns not to jump ahead into complex orchestration before the control-plane slice works.

AGENTS

1. The better framing

I would split Phase 0 into two design documents or two major sections of one design proposal:

LRH Storage Model
LRH Conversation Model

Each should be conceptual before it is implementation-specific.

LRH Storage Model

Defines:

privacy class
 durability class
 versioning class
 retention class
 location class
 promotion/export rules
 backend capabilities

LRH Conversation Model

Defines:

conversation schema
 turn schema
 attachment schema
 tool invocation schema
 conversation operations
 promotion operations
 external protocol primitives
 adapter boundaries
 security rules

This is better than prematurely saying “store chats in SQLite” or “store chats in Git.” SQLite, Git, local files, private control repos, MCP, and GitHub Apps become **backends and adapters**, not the conceptual foundation.

2. Storage model options

Option A — Backend-first storage

This says: choose SQLite now; design the API around it.

Pros

Fastest implementation path. SQLite is an excellent local durable store. It gives transactions, indexes, and reasonable local search. SQLite’s own atomic-commit documentation says that within a transaction, either all database changes occur or none occur. [SQLite](#) WAL mode also makes commits append records to a write-ahead log, allowing readers to continue from the original database while changes are committed. [SQLite](#)

Cons

It risks leaking implementation assumptions into the model. For example, conversations may become “rows in tables” rather than first-class LRH artifacts with promotion, retention, privacy, and versioning semantics.

Verdict

Good implementation backend. Bad conceptual starting point.

Option B — Path-first conceptual namespace

This is the idea you floated:

```
/transient/<group>/<session_id>/<turn>
/versioned/private/<group>/<session_id>/<turn>
/versioned/public/<group>/<session_id>/<turn>
/archived/private/<group>/<session_id>/<turn>
```

The namespace encodes policy. Similar in spirit to convention-over-configuration: location conveys retention, privacy, or durability.

Pros

Very LRH-friendly. It is human-readable, inspectable, and easy to explain. It lets you reason about state without querying a database. It also generalizes well across backends: a SQLite table, file tree, Git repo, or remote service can all implement the same conceptual paths.

This fits LRH's "human-auditable, machine-interpretable" approach. LRH's README says the framework turns a Git repository into a human-auditable, machine-interpretable control system using structured Markdown and metadata.

humans

Cons

Path-encoded policy can become brittle. Renaming or moving an object might accidentally change its privacy or durability semantics. If policy is only encoded in the path, you can lose history of why something was private, public, transient, retained, or expired.

Also, paths alone do not provide transactionality, indexing, concurrency, or retention enforcement. You still need a backing store and a policy engine.

Verdict

Excellent as a **conceptual address space**, but not sufficient as the only source of truth for policy.

Option C — Metadata-first storage model

Every stored object has explicit metadata:

YAML

```
id: TURN-123
privacy: private
durability: versioned
retention: keep
visibility: local
backend: sqlite
scope:
  project: replication-vector-control
  workstream: WS-STEAM-RELEASE
```

Pros

More explicit and less brittle than path-only policy. You can validate it, migrate it, query it, and preserve policy even if the object moves.

This also aligns with LRH's existing use of Markdown plus YAML frontmatter as human-readable source artifacts and structured machine-readable metadata.

humans

Cons

Requires metadata lookup. Slightly heavier. If every operation needs metadata resolution, simple file-based inspection becomes less obvious.

Verdict

Strong, but should be paired with human-readable paths.

Option D — Capability-first Storage API

This defines a storage interface around guarantees:

Python

```
put(object, policy)
append_event(stream, event, policy)
read(ref)
list(prefix, filters)
promote(ref, new_policy)
export(ref, target)
redact(ref, strategy)
seal(ref)
garbage_collect(policy)
```

The backend advertises capabilities:

```
supports_transactions
supports_versioning
supports_private_storage
supports_public_export
supports_retention
supports_encryption
supports_sync
supports_concurrent_writers
```

Pros

This is architecturally strongest. It lets LRH support SQLite, Git, local files, private control repos, and future remote services without pretending they all provide the same guarantees.

Cons

More design upfront. If overdone, this can become an enterprise storage abstraction before there is a single useful chat import command.

Verdict

Best conceptual foundation, as long as the first implementation is deliberately small.

3. Recommended storage design

Use a hybrid:

Conceptual namespace + explicit policy metadata + backend capability API

The path is the **human-facing address**.

The metadata is the **machine-validated policy**.

The backend capability API is the **implementation contract**.

For example:

`lrh://versioned/private/conversations/WS-STEAM-RELEASE/CHAT-2026-05-16/TURN-0004`

with metadata:

YAML

```
kind: conversation_turn
privacy: private
durability: versioned
retention: keep
authority: non_authoritative_context
backend: local_sqlite
exportable_to_repo: false
```

The namespace and metadata should agree. If they disagree, validation should fail.

4. Privacy, durability, versioning, and retention classes

I would define four independent axes.

Privacy class

```
private
shared_private
public
secret
```

Class	Meaning
private	local to user or private workspace
shared_private	safe for private control repo/team
public	safe for controlled public repo
secret	contains credentials or highly sensitive data; not exportable by default

Default should be **private** for raw conversations.

Durability class

```
transient
session
durable
versioned
sealed
```

Class	Meaning
transient	may be discarded automatically
session	survives current process/session but not guaranteed long-term
durable	should survive restart/crash under backend guarantees

Class	Meaning
versioned	changes are retained as history
sealed	immutable record, append-only references only

SQLite can implement durable event storage locally, but the exact durability depends on journaling/sync settings and filesystem behavior. The design should say “durable according to backend-advertised guarantees,” not make magical promises.

Retention class

```
ttl=<duration>
keep
archive_after=<duration>
delete_after=<duration>
manual
```

A path-like shorthand is useful:

```
/transient/ttl=7d/private/conversations/...
```

But retention should also be metadata so policy survives moves and exports.

Authority class

This is LRH-specific and important:

```
non_authoritative_context
draft
proposal
approved_run
evidence
authoritative_control_artifact
```

A chat turn should default to **non_authoritative_context**. A promoted work item or evidence record can become authoritative only through existing LRH mechanisms.

This matches LRH's control model: intent, execution, truth, and consequences are separate planes, and evidence/status are not simply asserted by conversation.

README

5. Should path encode policy?

Yes, but only as a **validated convention**, not the only policy source.

Recommended pattern:

```
lrh://<durability>/<privacy>/<domain>/<group>/<object-id>
```

Examples:

```
lrh://transient/private/conversations/local/CHAT-123
lrh://versioned/private/conversations/WS-STEAM-RELEASE/CHAT-456
lrh://versioned/public/evidence/WI-123/EV-789
lrh://sealed/shared_private/runs/RUN-456
```

Pros:

- readable
- grep-friendly
- portable across backends
- helps users reason about data placement
- supports policy-by-location

Required guardrail:

Path policy and metadata policy must match, and promotion must be an explicit operation.

So moving a file from **private** to **public** manually should not silently publish it. The storage API should require:

`promote(ref, target_policy)`

with checks/redaction/review.

6. The storage development tech tree

This gives a clear implementation path.

Storage Phase 0 — Design proposal

Define:

LRH URI scheme
policy axes
object metadata
backend capability model
promotion/export rules
privacy defaults
retention semantics

No code.

Storage Phase 1 — Filesystem prototype

Implement enough for manual import/export:

private local directory
JSON/Markdown objects
atomic writes
manifest
no concurrency promises beyond single-process

Storage Phase 2 — SQLite private ledger

Implement:

conversations
turns
attachments

- events
- run proposals
- indexes
- schema version
- migrations
- export
- backup

SQLite is a good fit here because of transactional semantics and local embedded deployment. [SQLite](#)

Storage Phase 3 — Private control repo export

Implement curated export:

- conversation summaries
- decision records
- prompt packages
- run records
- evidence records

Git stores the reviewed artifacts, not every raw turn.

Storage Phase 4 — Multi-repo control plane

Support one control repo coordinating multiple controlled repos.

This directly fits your Replication Vector example.

Storage Phase 5 — Team/server backend

Only after local semantics are stable.

7. Conversation model options

Now the second foundation: the conceptual Conversation API.

Conversation Option A — Transcript model

Conversation is a list of turns:

- conversation
 - turn
 - turn
 - turn

Pros

Simple. Mirrors chat UIs.

Cons

Too weak for LRH. It does not distinguish prompt drafts, tool calls, decisions, evidence, run proposals, or imports from external tools.

Verdict

Good base layer, insufficient alone.

Conversation Option B — Event-sourced conversation

Conversation is an append-only event stream:

```
conversation.created  
turn.added  
attachment.added  
summary.created  
proposal.created  
run.linked  
evidence.promoted  
conversation.sealed
```

Pros

Excellent for audit. Easy to preserve history. Fits conversation + execution flow. Supports imported external tool events.

Cons

Harder to read directly. Requires projection into user-facing views.

Verdict

Best underlying model.

Conversation Option C — Artifact graph

Conversation objects form a graph:

```
conversation → turn → prompt_draft → run_proposal → run → evidence → status
```

Pros

This maps beautifully to LRH's workflow. It makes promotion explicit.

Cons

Needs identity and references. Can become complex too early.

Verdict

Best conceptual model; implement gradually.

Conversation Option D — Conversation as context provider only

A conversation exists only to produce context packets for models. It is not a first-class artifact.

Pros

Simpler.

Cons

Misses the radical visibility goal.

Verdict

Not enough.

8. Recommended conversation model

Use:

event stream + artifact graph

The event stream preserves what happened.

The artifact graph explains what it means.

Core objects:

Conversation
ConversationTurn
ConversationEvent
Attachment
ContextPacket
ModelRequest
ModelResponse
ToolInvocation
PromptDraft
RunProposal
PromotionRecord
EvidenceCandidate
ConversationSummary

Core operations:

```

create_conversation(scope, policy)
append_turn(conversation, role, content, metadata)
attach(conversation_or_turn, artifact)
summarize(conversation)
create_prompt_draft(conversation)
create_run_proposal(conversation)
promote(ref, target_kind)
export(ref, target_backend)
import_external(source)
seal(ref)
redact(ref)

```

The key rule:

A conversation can generate a proposal, but cannot by itself mutate authoritative LRH state.

9. Conversation scopes

A conversation should have explicit scope:

```
global
workspace
project
repository
workstream
work_item
run
evidence
```

Your multi-repo control-plane example makes this important. A Steam-release conversation might be scoped to:

YAML

```
scope:
  type: product_release
  control_repo: replication-vector-control
  controlled_repos:
    - velumin
    - replication-vector-engine
    - replication-vector-assets
    - steam-publishing
```

This lets LRH represent one control plane coordinating several repos without confusing “where the code lives” with “where the work is governed.”

10. Conversation API vs external protocol

The Conversation API is internal/conceptual. External protocol surfaces should be adapters.

Internal API

Stable conceptual operations:

```
conversation.create
conversation.append_turn
conversation.attach
conversation.summarize
conversation.promote
run.propose_from_conversation
evidence.attach_from_conversation
```

CLI adapter

Bash

```
lrh conversation create
lrh conversation append
lrh conversation import
lrh conversation summarize
lrh conversation promote
```

File adapter

inbox/conversation-import.json
outbox/conversation-summary.json

MCP adapter

MCP resources can expose conversation/context objects, and MCP tools can expose controlled operations. MCP's resource spec says resources allow servers to share data that provides context to language models, and each resource is identified by a URI. [Model Context Protocol](#) MCP prompts also let servers expose reusable prompt templates and workflows to clients. [Model Context Protocol](#)

GitHub adapter

Slash commands, PR links, review comments, and run/evidence imports.

The API should be stable even if adapters come and go.

11. Security model

The design should borrow a strong principle from OWASP's LLM prompt-injection guidance: separate untrusted content from privileged action. OWASP describes a strong form of this as a privileged model

[OWASP Cheat Sheet](#)

holding tools while a quarantined model reads untrusted content but cannot take action. [Series](#)

For LRH, translate that into:

conversation text is untrusted
conversation summaries are reviewed/derived
run proposals are explicit
execution requires approval/guardrail checks
evidence is attached, then interpreted
status is derived, not declared

Security decisions:

Risk	Design response
prompt injection	imported text cannot authorize actions
secret leakage	private by default; public promotion requires review
accidental publication	explicit promote/export operation
unsafe tool execution	run proposal and guardrail review
spoofed external agent	provenance and adapter identity
evidence forgery	source links, hashes, logs, validation
storage loss	backend durability capability + backup/export

This also matches LRH's existing emphasis that status should be grounded in evidence rather than optimistic summaries.

AGENTS

12. Options for Phase 0 design effort

Option 1 — Keep Phase 0 lightweight

Write one design proposal with broad sections:

storage tiers
conversation schema
interop primitives
security

Pros

Fast. Low process overhead. Fits LRH's small-change discipline.

Cons

May not resolve the key conceptual issues. Implementation may later drift.

Verdict

Good if you want just enough design to start.

Option 2 — Two explicit conceptual designs

Create:

storage_model.md
conversation_model.md

or one proposal with two major parts.

Pros

Best balance. It steelmans both concerns without implementation bloat.

Cons

More design work before code.

Verdict

Recommended.

Option 3 — Full protocol RFC

Define detailed schemas, commands, backend capability matrices, and adapter protocols.

Pros

Very clear for future implementers and third-party tool authors.

Cons

Too much too soon. Risks freezing abstractions before dogfooding.

Verdict

Later, after prototype feedback.

Option 4 — Prototype first, design later

Build `lrh conversation import` and a SQLite store, then document what emerges.

Pros

Fast feedback.

Cons

High risk of baking in the wrong abstractions. Storage/privacy mistakes are hard to unwind.

Verdict

Not ideal for this domain.

13. Decisive advantage

The decisive advantage belongs to **Option 2: two explicit conceptual designs before implementation.**

Specifically:

Design A: LRH Storage Model
Design B: LRH Conversation Model

This gives enough rigor to avoid bad foundations, but not so much that LRH gets trapped in a giant spec effort.

It also matches LRH's own style guide: small, scoped, clear, deterministic, and reviewable changes are preferred over broad rewrites or speculative implementation.

STYLE

14. Disqualifying limitations

Disqualify path-only policy

Encoding policy in paths is useful, but path-only policy is too brittle. Metadata must confirm it.

Disqualify Git-only storage for raw conversations

Git is good for curated artifacts and private control repos. It is not the right live database for raw, high-volume, sensitive, append-heavy chat state.

Disqualify public-by-default chat storage

Raw conversations should be private unless explicitly promoted/exported.

Disqualify conversation-as-authority

A chat should not be able to mark work complete, alter status, or authorize runs. It can propose artifacts; LRH validates and promotes them.

Disqualify implementation-before-policy

For conversations, storage, and agents, privacy and durability semantics should exist before a backend implementation.

15. Recommended Phase 0 shape

I would write the next design proposal as:

LRH Conversations, Storage, and External Agent Interop

with two core sections:

Part 1 — LRH Storage Model

- Goals and non-goals
- Conceptual LRH URI namespace
- Privacy classes
- Durability classes
- Versioning classes
- Retention classes
- Authority classes
- Storage object metadata
- Backend capability model
- Promotion/export rules
- Private control repo pattern
- Failure modes and guarantees
- Open questions

Part 2 — LRH Conversation Model

- Goals and non-goals
- Conversation scopes
- Conversation schema
- Event stream model
- Artifact graph model
- Conversation operations
- Promotion path
- External protocol primitives
- Adapter surfaces: CLI, file, MCP, GitHub, future UI

Security model
Open questions

And one crucial statement:

LRH Conversations are non-authoritative context until promoted into normal LRH control artifacts such as design proposals, work items, run proposals, evidence, or status through explicit operations.

Final recommendation

Yes: go a notch deeper before implementation.

The best architecture is:

storage model first
conversation model second
adapters third
implementation fourth

More concretely:

Conceptual Storage API
→ private durable ledger backend
→ curated repo exports
→ private control repo support

Conceptual Conversation API
→ CLI/file adapters
→ MCP adapter
→ GitHub adapter
→ LRH serve chat UI

This preserves the best of both worlds: rigorous LRH control-plane semantics and openness to tomorrow's better external tools.

Yes, this looks really good. And I want to record it as a design document. But I think we need to interrogate one more thing: the end user use case or tool use case. That is, if we build this, what features are enabled by it?

There are several reasons I want chat:

- it would be super to have a dialogue with ChatGPT and for there to be a LRH GPT which I could say, "great, <at>lrh record this as a design proposal" or "great, <at>lrh run this work item".
- it would be even MORE super to open the LRH meta dashboard, look at the swimlane for "needs attention", pick a project, and start a chat, with the chat talking to ANY backend that supported it legitimately, or to a tool we develop ourselves, and then just decide what to do and kick off the next round - even if the LRH meta dashboard wasn't where a particular developer "lived" most of their workday, it would be SUPER convenient.
- it would be EVEN MORE SUPER if a Github, Github Speckit, OpenAI, Anthropic or Aider representative called and said, "Oh hey, our teams have started using the LRH control plane, we'd love to do an integration into our feature so we can kick off run items from within our tool" and we said "sure thing, here's the API surface that you want, now let's wire it up."

So what I'm thinking of is let's outline 3-5 clear use cases for this feature, and identify for each:

- what are the pros of having this feature? what does it enable?
- what are the cons of this feature? the technical risks, security risks, or costs?

- what are the technical requirements of this feature? how much build do we need to get bang for our buck?
- what are the existing alternatives to this feature? how much can we get by using some other tool and integrating more simply?

As a brief for instance, I just checked my wallet and found 84 bucks (and change). That, plus a billion dollars would enable me to train an LRH-specific LLM chatbot. However, I could do other things with that billion dollars, such as retire from programming and work on my writing and art. Also, I don't have that billion dollars. So I'm thinking building out an LRH-specific LLM chatbot isn't the best starter plan and that a continued use of the OpenAI ChatGPT subscription is a better use of our money. However, conversely, there is no open-source, user-centric AI software development dashboard that puts me in control of my own data; even though <https://kiro.dev/> exists, it's putting my software controls into someone else's silo. Logical Robotics aspires to build what every software developer doesn't yet know they need, and no-one building a siloed system has thought to try to build: an agentic software development toolkit that puts the developer in as much control of their own data as they choose to exercise.

What use cases would the proposed conversation design enable, and which ones are a good fit for LRH? Are there enough of them to greenlight this feature?

Yes — there are enough strong use cases to greenlight **Phase 0 design work** for LRH Conversations, Storage, and External Agent Interop.

There are **not** yet enough reasons to greenlight a full LRH-native agentic chatbot, a custom LRH-trained LLM, or a heavyweight hosted platform. The right next step is a **design proposal that makes the use cases, storage model, conversation model, and external API surfaces explicit**.

The core thesis is:

LRH Conversations are valuable because they let developers keep using the best available AI tools while moving the durable workflow record, project context, run proposals, evidence, and status interpretation back under user control.

That is very LRH. LRH's own framing is that it turns repositories into human-auditable, machine-interpretable control systems and supports AI-assisted work through explicit artifacts rather than hidden context.

humans

Bottom line

I would greenlight this as a **design stream** and then a **small storage/conversation prototype**, not as a full chatbot product.

The best starter feature is not “train an LRH LLM.” It is:

Capture, scope, store, summarize, promote, and export conversations.

The best strategic feature is:

Let external tools participate in LRH through stable storage and conversation/run/evidence protocols.

The most important product insight is that the LRH value is not the model. The value is the **control plane around the model**.

Use case 1 — “@lrh record this” from ChatGPT/Codex/Claude/etc.

What the user wants

You are in ChatGPT, Codex, Claude, or another tool. The conversation reaches a useful result. You want to say something like:

```
@lrh record this as a design proposal
@lrh create a work item from this
@lrh create a prompt package from this
@lrh propose a run for WI-123
```

This could start as a copy/paste workflow, then become a Custom GPT Action, MCP tool, browser extension, local CLI bridge, or future vendor integration.

OpenAI’s GPT Actions are relevant here because they allow a Custom GPT to connect to third-party services through defined actions and API schemas. [OpenAI Developers+1](#) MCP is also relevant because it is an open protocol for connecting LLM applications to external data sources and tools. [Model Context Protocol](#)

What it enables

This is probably the clearest near-term “wow” feature.

It lets you keep using best-of-breed tools while converting useful dialogue into LRH artifacts:

```
chat → design proposal
chat → work item
chat → prompt package
chat → run proposal
chat → evidence note
chat → decision record
```

That directly addresses the hidden-context problem. Instead of a useful conversation dying inside a vendor app, LRH can capture the outcome in a durable, scoped, auditable form.

Pros

It is user-centric. It does not force the user to abandon ChatGPT, Codex, Claude, or tomorrow’s NovumAI. It also has a low-friction adoption path: initially, the user can copy/paste; later, external integrations can call a formal LRH API.

It fits LRH’s prompt workflow philosophy. PROMPTS.md says prompt IDs and execution records exist to make prompt-driven work easier to trace, search, and review.

PROMPTS

Cons / risks

The main risk is accidental authority. A chatbot saying “record this as complete” should not mean the work is complete. Conversation output must be treated as a **draft or proposal** until promoted through LRH’s normal rules.

There are also privacy risks. A ChatGPT conversation may include sensitive client material, credentials, unpublished ideas, or proprietary code. Raw conversation capture must default to private storage.

Technical requirements

Minimum useful build:

```
LRH storage API
conversation object
conversation import/capture
conversation summary
promotion operation
manual CLI flow
```

Example commands:

Bash

```
lrh conversation import transcript.md --scope work-item --id WI-123
lrh conversation summarize CHAT-123
lrh conversation promote CHAT-123 --to design-proposal
```

Higher-value later build:

GPT Action / MCP adapter / browser extension / local deep-link

Existing alternatives

The alternatives are manual copy/paste into Markdown, ChatGPT custom GPTs without LRH storage, Codex task records, Claude project chats, Notion/Google Docs notes, or GitHub issue comments.

These alternatives can capture text, but they do not naturally bind the conversation to LRH scope, evidence, prompt IDs, work items, run proposals, and status.

Fit for LRH

Excellent. This should be greenlit early.

Use case 2 — Contextual chat inside the LRH meta/project dashboard

What the user wants

Open the LRH meta dashboard, look at the “Needs Attention” swimlane, select a project, and start a scoped chat:

```
Why is this project in Needs Attention?
What should I do next?
```

Draft a work item for this issue.
Summarize the current blockers.
Create a run proposal for the next safe action.

The backend could be OpenAI, Anthropic, local models, Aider-like tools, or a future provider.

What it enables

This makes LRH a daily workbench rather than just a validator or document model.

The value is not that LRH has the best chat UI. The value is that the chat is already scoped to:

project
workstream
work item
current focus
evidence
validation status
guardrails
recent runs

That context is what makes the interaction valuable. LRH's README describes `lrh snapshot` as the primary interface for humans and AI agents because it produces current focus, active work items, interpreted status, warnings, and inconsistencies. A dashboard chat panel would be a natural conversational layer over that same snapshot. humans

Pros

It reduces app switching. It makes "what should I do next?" a direct workflow. It also makes the meta dashboard actionable: the user can triage, discuss, draft, and propose next steps from one place.

This use case is especially good for multi-project control planes. It makes the dashboard more than a pretty list of statuses.

Cons / risks

This is more technically demanding than import/capture. It needs:

provider abstraction
prompt/context builder
storage API
conversation scoping
privacy defaults
cost visibility
model configuration
auth/secrets handling

It also risks becoming an "AI chat product" if not bounded. The first version should be non-agentic: ask, summarize, draft, propose. Not execute.

Technical requirements

Minimum useful build:

context packet builder
conversation storage

model provider adapter
chat panel or CLI command
promotion to draft artifacts

Example:

Bash

```
lrm conversation ask --project LRH --work-item WI-123 "What is blocking this?"
```

Dashboard version:

Select Project → Open Chat → Ask → Save summary → Promote to work item/run proposal

Existing alternatives

ChatGPT Projects, Claude Projects, Kiro, Aider, Cursor, and IDE chat tools can all provide contextual assistance. Kiro, for example, advertises multimodal chat, spec-driven development, agent hooks, and context management. [Kiro+1](#) Aider provides AI pair programming in the terminal and supports in-chat commands and chat modes. [Aider](#)

But those tools do not make LRH's project control plane the durable source of scope, evidence, status, and promotion.

Fit for LRH

Excellent, but not first implementation. This is a strong Phase 2 or Phase 3 after the storage and conversation APIs exist.

Use case 3 — External tools integrate with LRH as a control-plane protocol

What the user wants

A tool vendor or open-source project says:

We want to integrate with LRH.
How do we get project context?
How do we propose a run?
How do we attach evidence?
How do we report completion?
How do we let the user approve?

This is the "if GitHub/OpenAI/Anthropic/Aider calls" scenario.

What it enables

This is the most strategically important use case.

It means LRH can become a **protocol and control plane** rather than one more silo. External tools could call into LRH to:

```
read scoped context
request a snapshot
propose work
create a run proposal
append progress events
attach evidence
link a PR
request status interpretation
```

MCP is a natural candidate surface here because its resources allow servers to expose application-specific context by URI, and its tools allow clients/models to invoke external capabilities. [Model Context Protocol+1](#)

Pros

This gives LRH the best shot at long-term relevance. If tomorrow's best AI development tool is not LRH, that is fine: it can still participate in LRH's workflow.

This also fits LRH's architecture boundary. LRH is supposed to be reusable across independent project repositories and should not be hard-coded to its own repo. A protocol-first design supports that.

AGENTS

Cons / risks

Protocol design is hard. Too little structure creates chaos; too much creates a premature standard nobody uses.

Security is the biggest risk. External tools must not be able to silently mutate authoritative project state, mark work complete, or execute dangerous actions.

Technical requirements

Minimum useful build:

```
stable JSON schemas
CLI commands with --format json
context packet schema
run proposal schema
evidence attachment schema
conversation import/export schema
```

Later:

```
MCP server
GitHub App adapter
OpenAPI endpoint
vendor SDK examples
```

Existing alternatives

MCP already provides a general tool/resource protocol. GitHub Apps provide a way to integrate with PRs, issues, permissions, and webhooks. GPT Actions provide a way for a Custom GPT to call external APIs.

[Model Context Protocol+1](#)

LRH should not replace those standards. It should define **LRH-specific resources and operations** that can be exposed through CLI, file exchange, MCP, GitHub, or HTTP.

Fit for LRH

Very strong. This should be part of the design from the beginning, even if implementation comes later.

Use case 4 — Durable private conversation ledger with curated repo promotion

What the user wants

The user wants their development conversations preserved, searchable, and linkable to work without dumping raw private transcripts into public repos.

For example:

Raw chat: private durable local store
Curated summary: private control repo or project context
Run proposal: project/executions or run ledger
Evidence: project/evidence
Status: derived by LRH

What it enables

This is the foundation for everything else. Without a durable storage model, all other conversation features become fragile.

It enables:

search old conversations
recover context after reboot
summarize prior decisions
link chats to work items
promote useful outcomes
avoid public leaks
coordinate multiple repos

It also supports the private-control-repo scenario for multi-repo products like Replication Vector.

Pros

This gives the user control over their data. That is the key differentiator versus siloed systems.

It also aligns with LRH's separation between authoritative project artifacts and runtime/private state. LRH docs already distinguish human-readable Markdown artifacts from runtime structured objects and private/global state in the meta workspace.

README

Cons / risks

Storage design mistakes are expensive. Privacy mistakes are especially expensive.

If raw chat storage is public by default, that is a serious failure. If Git is the only backing store for high-frequency raw chat, that is also a poor fit. If SQLite is used without export, it becomes another opaque

silo.

Technical requirements

Minimum useful build:

storage API
object metadata
privacy/durability/retention classes
local private backend
export to Markdown/JSON
redaction markers
backup/export command

The backend could start as file-based, but a SQLite ledger is probably the best real local durable backend because SQLite supports transactional atomic commit. [Model Context Protocol](#)

Correction: that last citation is MCP, not SQLite. The storage design should cite SQLite separately before implementation, but the general architectural point remains: use a backend with explicit durability guarantees rather than casual file writes.

Existing alternatives

ChatGPT and Claude retain conversation histories inside their apps. IDE tools and coding agents may maintain their own logs. GitHub PRs retain review conversation. Notion/Obsidian can preserve notes.

But none of these gives you a unified LRH-scoped, privacy-tiered, promotion-aware conversation ledger.

Fit for LRH

Essential. This is the foundation use case.

Use case 5 — Multi-repo product/release control plane

What the user wants

For Replication Vector:

Velumin graphics library repo
Replication Vector game engine repo
private asset/level repo
private Steam publishing repo
one LRH control plane

The user wants conversations, decisions, work items, evidence, and release status to coordinate across all of them.

What it enables

This is a major differentiator. Many real products are not one repo. LRH can become the control layer that coordinates:

public code
private code
assets
publishing
compliance
release evidence
marketing or store metadata

The conversation model becomes useful because release discussions can be scoped to the product, a repo, a workstream, a work item, or a specific run.

Pros

This is a high-value “adult workflow” that many AI coding tools do not handle well. It also reinforces the need for a private control repo pattern.

It fits LRH’s architecture: the harness code is reusable, client project repositories have their own **project/** directories, and LRH should not be hard-coded to a single repo.

AGENTS

Cons / risks

Cross-repo identity, permissions, and evidence linking are hard. You need stable references:

repo id
project id
workstream id
work item id
run id
evidence id
external PR/link id

You also need a careful privacy model because some repos are public and some are private.

Technical requirements

Minimum useful build:

control repo concept
controlled repo registry
cross-repo references
storage privacy classes
conversation scope model
evidence links across repos

Existing alternatives

GitHub Organizations, GitHub Projects, Linear, Jira, Notion, and release checklists can coordinate multi-repo work. Game studios often use project-management tools plus version-control systems like Git or Perforce.

But these tools usually do not provide LRH’s evidence-backed, AI-agent-readable control plane.

Fit for LRH

Very strong, but later than local conversation capture. It should influence the storage model now.

Use case 6 — Agentic run cockpit: chat → run → PR → review → evidence → status

What the user wants

From a conversation, propose and run the next work item:

Draft prompt
Select agent/tool
Approve run
Track run events
Link branch/PR
Import review
Attach evidence
Interpret status

What it enables

This is the “Huge Loop” operationalized. It would make LRH a visible control system for agent-assisted development, not just a documentation framework.

This maps directly onto LRH’s action lifecycle: Work Item → Action Proposal → Guardrail Review → Action Decision → Execution → Evidence → Status.

[README](#)

Pros

High leverage. This is the workflow developers will eventually want once conversation capture and project context work.

Cons / risks

This is where LRH becomes operationally dangerous if done wrong. It touches tokens, code changes, shell commands, PRs, credentials, and external tools.

Prompt injection and unsafe tool use are major concerns. OWASP’s prompt-injection guidance describes the risk of untrusted text manipulating LLM behavior and causing unintended actions or data disclosure.

[WIRED](#)

Technical requirements

Significant:

run proposal schema
guardrail approval
tool adapter permissions
event log
evidence import
PR integration
failure/retry model
status interpretation

Existing alternatives

Codex, Copilot coding agent, Claude Code, Aider, OpenHands, Kiro, and similar tools already perform pieces of this. GitHub's Copilot coding agent, for example, can work in a GitHub-hosted environment and produce code changes that can become PRs. Kiro offers spec-driven development and agent hooks.

[Kiro+1](#)

Fit for LRH

Excellent as a mid-term target, not as the first implementation.

Which use cases are good fits for LRH?

Strong fits

These are very good LRH use cases:

1. @lrh record/promote from external chat
2. Durable private conversation ledger
3. Contextual dashboard chat
4. External tool protocol
5. Multi-repo private control plane
6. Chat-to-run cockpit

But they should not be built in that order.

What should be greenlit?

Greenlight now: design

Yes. There are enough use cases to greenlight a design document.

The design should cover:

storage model
conversation model
privacy/durability/retention/authority classes
conversation API
promotion API
external protocol surfaces
multi-repo control repo pattern
security model
starter feature set

Greenlight next: small prototype

After design, greenlight a small prototype only if it is narrowly scoped:

local private conversation capture
manual import/export
conversation summary record
promotion to design/work-item/prompt draft

Do not greenlight yet

Do not yet greenlight:

full LRH chatbot
custom LRH-trained LLM
hosted team service
direct agent execution from chat
complex MCP/GitHub integrations before storage and promotion semantics

Bang-for-buck ranking

Rank	Use case	Build size	Strategic value	Starter fit
1	Durable private conversation ledger + curated export	Medium	Very high	Excellent
2	@lrh record/promote this	Small/medium	Very high	Excellent
3	External protocol primitives	Medium	Very high	Excellent design, later implementation
4	Contextual dashboard chat	Medium/high	High	Good after storage
5	Multi-repo control repo	Medium/high	High	Design now, implement later
6	Chat-to-run cockpit	High	Very high	Later
7	LRH-specific trained chatbot	Enormous	Unclear	Poor starter

The LRH-specific LLM is the wrong starter because model quality is not the missing primitive. Control, provenance, context, storage, promotion, and evidence are the missing primitives.

Existing alternatives and what LRH should learn from them

Existing tool	What it already does well	What LRH should not try to clone first	What LRH can add
ChatGPT / Claude	general reasoning and chat	model UX	durable LRH-scoped promotion
Codex / Copilot	code-oriented agent work	hosted agent execution	run/evidence/status control plane
Aider	local repo chat + edits	terminal pair-programming UX	LRH artifact promotion and evidence tracking
Kiro	spec-driven IDE, hooks, context	full IDE	open control-plane protocol and storage

Existing tool	What it already does well	What LRH should not try to clone first	What LRH can add
GitHub	PRs, reviews, issue comments	project reasoning layer	LRH interpretation and evidence synthesis
Notion/Obsidian	notes and knowledge base	agentic control loop	typed LRH conversation/run schema

Aider's docs describe it as AI pair programming in the terminal, including in-chat commands and chat modes. [Aider](#) Kiro advertises multimodal chat, spec-driven development, agent hooks, and smart context management. [Kiro](#) These tools validate the demand. LRH's opportunity is to avoid becoming another silo.

Recommended use-case-driven design outline

I would structure the design document around the use cases, not only the architecture:

LRH Conversations, Storage, and External Agent Interop

Summary

Motivation

- Hidden chat context problem
- User control over AI development data
- External tools should plug into LRH, not be replaced by LRH

Use Cases

1. @lrh record/promote this
2. LRH dashboard contextual chat
3. External tool integration API
4. Durable private conversation ledger
5. Multi-repo private control plane
6. Chat-to-run cockpit

Non-goals

- Custom LRH-trained LLM
- Full chatbot clone
- Raw public transcript storage by default
- Direct unapproved agent execution

Storage Model

- privacy, durability, retention, authority
- local private ledger
- curated public/private repo exports
- private control repo pattern

Conversation Model

- conversation, turns, events, attachments
- scopes
- summaries
- prompt drafts
- run proposals
- evidence candidates

Promotion Model

conversation → summary → proposal → run → evidence → status

External Protocol Model

- CLI
- file exchange
- MCP
- GPT Actions/OpenAPI
- GitHub App/slash commands
- future UI

```
## Security Model
- private by default
- untrusted input boundaries
- explicit promotion
- human approval for runs
- no direct status mutation
```

```
## Phased Implementation
```

Final answer

Yes, there are enough use cases to greenlight the feature — **as a design and staged capability**, not as a full chatbot.

The killer use cases are:

1. **Capture/promote useful external chats into LRH artifacts.**
2. **Maintain a private durable conversation ledger under user control.**
3. **Provide contextual chat from the LRH dashboard.**
4. **Expose a protocol so external tools can plug into LRH.**
5. **Coordinate multi-repo products through a private control plane.**
6. **Eventually drive the chat → run → evidence → status loop.**

The starter plan should be:

```
Design now.
Build storage + import/export + promotion first.
Add contextual chat second.
Add external protocol adapters third.
Add chat-to-run only after guardrails are mature.
```

That gives LRH a realistic path to something genuinely differentiated: **an open, user-controlled, evidence-backed AI software development control plane that can use any good agent tool without surrendering the developer's data or process to that tool's silo.**

Thanks. This is a good analysis, combined with our previous overall design. Please flesh out a detailed, unified design to implement this cluster of ideas; once we've fleshed it out we will then record it as a design proposal.

Review the idea, review recommended best practices in the area, generate a high-level design, identify the low-level choices needed, consider the pros and cons, and evaluate them with respect to the best practices. Use best practice engineering and architectural ideas for the relevant kind of idea being implemented. Use your best judgment on neither cutting corners or incurring technical debt, nor over-engineering prematurely or unnecessarily.

If key information is missing, stop and ask for clarification.

If there is a serious issue that needs to be addressed, also stop and alert the user.

Otherwise, if there is enough information to proceed, outline a design to implement the idea in a best-practices way.
Let's think step by step and check our work to make sure we get the right answer, and cite or quote reputable sources in our answer so we know it is grounded.

There is enough information to proceed. I do not see a serious blocker, but there is one hard constraint to put at the top of the design:

Conversations must not become an authority layer. They are context, provenance, deliberation, drafts, proposals, and evidence candidates. They become authoritative only when explicitly promoted into existing LRH artifacts such as design proposals, work items, execution records, evidence, or derived status.

That single rule keeps the idea aligned with LRH's control-plane model instead of creating a parallel "chat said so" system.

Below is a unified design you can later record as a proposal.

LRH Conversations, Storage, and External Agent Interop

1. Summary

This design introduces **LRH Conversations**: a transparent, user-controlled, vendor-neutral conversation and agent-interoperability layer for LRH.

The goal is not to build "LRH ChatGPT." The goal is to let developers keep using ChatGPT, Codex, Claude, Aider, GitHub Copilot, Kiro-like tools, future tools, and LRH-native tooling while making the development dialogue:

scoped
stored
searchable
private by default
promotable into LRH artifacts
linkable to runs and evidence
usable by external tools through stable protocols

The design has two foundations:

1. LRH Storage Model
2. LRH Conversation Model

External adapters — CLI, file exchange, MCP, GitHub App, GPT Actions/OpenAPI, and future `lrh serve` chat UI — sit on top.

This fits LRH because LRH is already defined as a repository-centered control plane for "designing, executing, and evaluating structured, evidence-backed, agent-assisted work," and its project artifacts are intended to be explicit, interpretable, actionable, and auditable.

humans

2. Motivation

A large portion of real AI-assisted development happens in opaque chats:

ChatGPT
Codex
Claude Code
GitHub Copilot
Aider
Kiro-like tools
future tools

These tools are useful, but their conversations are often siloed. The project may retain a PR, issue, work item, or final patch, but not the deliberation that produced it.

That conflicts with LRH's purpose: make project state, execution, evidence, and status visible, inspectable, and grounded. LRH's AGENTS guidance explicitly says that status should be grounded in evidence and not in optimistic summaries detached from tests, logs, metrics, screenshots, reports, or review notes.

AGENTS

The proposed feature cluster solves this by making conversations part of the LRH workflow without making them authoritative.

3. Target use cases

Use case 1 — “@lrh record this”

A user is in ChatGPT, Codex, Claude, Aider, GitHub, or another tool and says:

@lrh record this as a design proposal
@lrh create a work item from this
@lrh create a prompt package
@lrh propose a run for this work item

What it enables

Useful discussion can be promoted into durable LRH artifacts rather than dying inside an app-specific chat history.

Requirements

Minimum:

conversation import
conversation summary
promotion operation
storage policy
artifact draft generation
manual review

Later:

MCP tool
GPT Action/OpenAPI endpoint
GitHub slash command
browser/desktop bridge

Best fit

This is one of the best starter use cases because it gives immediate value while avoiding direct agent execution.

Use case 2 — LRH dashboard contextual chat

A user opens the LRH meta dashboard, selects a project in “Needs Attention,” and starts a scoped chat:

```
Why is this project in Needs Attention?  
What is the next safe action?  
Draft a work item.  
Propose a run.  
Summarize the evidence gap.
```

What it enables

The dashboard becomes an operational workbench. The chat is useful because it is automatically scoped to LRH context: project, workstream, work item, focus, validation, evidence, and guardrails.

Requirements

```
context packet builder  
model provider adapter  
conversation storage  
privacy defaults  
cost visibility  
promotion flow  
read-only/non-agentic first version
```

Best fit

Strong, but after the storage and conversation APIs exist.

Use case 3 — External tool integration API

A vendor or open-source tool wants to integrate:

```
give me scoped LRH context  
let me propose a run  
let me report progress  
let me attach evidence  
let me link a PR  
let LRH interpret status
```

What it enables

LRH becomes a control-plane protocol rather than a silo. Tomorrow’s best AI tool can plug into LRH instead of replacing it.

MCP is especially relevant here because it defines a standardized way for applications to provide resources and tools to LLM systems; MCP resources expose application-controlled context by URI, and

[OWASP Cheat Sheet](#)

tools expose operations clients can invoke. [Series](#)

Requirements

stable JSON schemas
CLI JSON commands
file exchange protocol
MCP resources/tools later
GitHub adapter later
permission model

Best fit

Strategically central. Design now; implement incrementally.

Use case 4 — Durable private conversation ledger

A user wants raw conversations preserved privately and searchable, while curated summaries or promoted artifacts can be committed to a public or private control repo.

What it enables

User control over AI development process data.

Requirements

storage API
privacy classes
durability classes
retention classes
local durable backend
export/import
redaction markers
backup/export command

Best fit

Foundational. This should come before any serious chat UI.

Use case 5 — Private multi-repo control plane

A product spans multiple repos:

public graphics library
public or private game engine
private assets and levels
private Steam publishing information
one private LRH control repo

What it enables

One LRH control plane can coordinate many repositories, with appropriate privacy boundaries.

Requirements

control repo concept
controlled repo registry
cross-repo references
scope model

private/public storage split
evidence links across repos

Best fit

Design now. Implement after storage and meta/workspace semantics mature.

Use case 6 — Chat-to-run cockpit

A conversation produces a run proposal:

chat → prompt draft → run proposal → guardrail review → execution → evidence → status

What it enables

The “Huge Loop” becomes visible and controlled.

This maps naturally to LRH’s action lifecycle: **Work Item** → **Action Proposal** → **Guardrail Review** → **Action Decision** → **Execution** → **Evidence** → **Status**.

[README](#)

Requirements

run proposal schema
approval workflow
adapter permissions
event log
evidence import
PR integration
failure/retry model
status interpretation

Best fit

Excellent mid-term target, not first implementation.

4. Design principles

4.1 Conversations are non-authoritative by default

A conversation can produce:

drafts
summaries
recommendations
proposals
evidence candidates

It cannot directly produce:

completed work status
approved run
accepted evidence
authoritative roadmap update

Those require explicit promotion into LRH artifacts and validation.

This follows LRH's distinction between human-readable source documents and structured runtime objects, and its warning not to treat raw dictionaries or unstructured data as long-term internal APIs when typed models are appropriate.

AGENTS

4.2 Private by default

Raw conversations should default to private, local durable storage.

Public or shared artifacts require explicit promotion/export.

This is not merely UX caution. Conversation records may contain secrets, private client material, unreleased product details, credentials, proprietary code, or untrusted text.

4.3 Durable when promised, transient when marked

The storage model should never imply durability accidentally.

Every object should declare a durability class:

```
transient
session
durable
versioned
sealed
```

If an object is **transient**, users should know it may disappear. If it is **versioned** or **sealed**, LRH must either provide the guarantee or reject the backend.

SQLite is a strong local backend candidate because it is an embedded database with transactional guarantees; SQLite describes itself as implementing ACID transactions even across program, OS, or power failures. [SQLite](#)

4.4 External tools propose; LRH decides

External tools may submit:

```
context requests
conversation imports
run proposals
progress events
evidence candidates
PR links
```

They should not silently mutate authoritative state.

This matters because LLM applications are vulnerable to prompt injection. OWASP describes prompt injection as malicious input that manipulates model behavior because instructions and data are

[OWASP Cheat Sheet](#)

processed together without clear separation. [Series](#)

4.5 Git is for curated history, not raw chat logs

Git is excellent for curated, reviewable artifacts. The Git book defines version control as recording changes to files over time so specific versions can be recalled later. [Git](#)

But raw conversations are high-volume, sensitive, append-heavy, searchable, and sometimes need redaction. They are better stored first in a private ledger, then curated into Git.

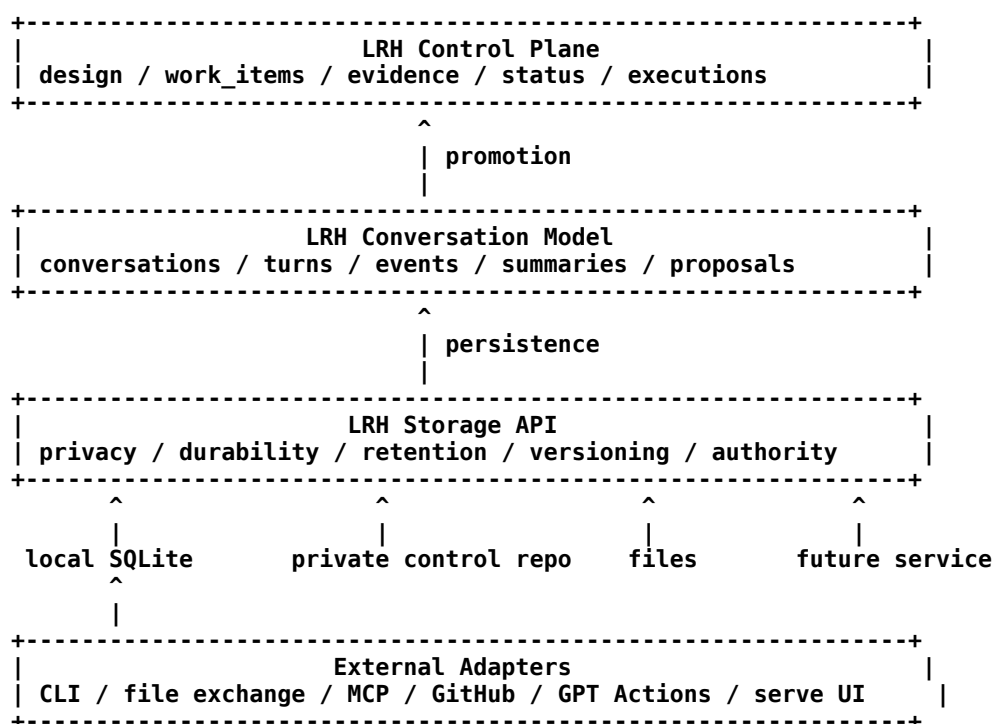
Recommended pattern:

raw private ledger → curated export → private/public control repo commit

not:

every chat turn → Git commit

5. High-level architecture



6. LRH Storage Model

6.1 Conceptual address space

Use an LRH URI-like address scheme:

`lrh://<durability>/<privacy>/<domain>/<scope>/<object-id>`

Examples:

```

lrh://transient/private/conversations/local/CHAT-123
lrh://versioned/private/conversations/WS-STEAM-RELEASE/CHAT-456
lrh://versioned/public/evidence/WI-123/EV-789
lrh://sealed/shared_private/runs/RUN-456
  
```

This gives users a readable mental model.

But path policy must not be the only policy. Each object also has metadata. Path and metadata must agree.

6.2 Storage policy axes

Privacy

private
shared_private
public
secret

Privacy	Meaning
private	local or personal workspace only
shared_private	private team/control repo eligible
public	public repo export eligible
secret	not exportable by default

Default for raw chat: **private**.

Durability

transient
session
durable
versioned
sealed

Durability	Meaning
transient	may be discarded automatically
session	expected to survive current session only
durable	expected to survive restart/crash under backend guarantees
versioned	prior versions/events retained
sealed	immutable finalized record

Retention

ttl=<duration>
keep
archive_after=<duration>
delete_after=<duration>
manual

Authority

non_authoritative_context
draft
proposal
approved_run
evidence_candidate

accepted_evidence
authoritative_control_artifact

This LRH-specific axis prevents chat from bypassing the control plane.

6.3 Object metadata

Every stored object should have metadata resembling:

YAML

```
id: TURN-0004
kind: conversation_turn
uri: lrh://versioned/private/conversations/WS-STEAM-RELEASE/CHAT-456/TURN-0004
privacy: private
durability: versioned
retention: keep
authority: non_authoritative_context
created_at: 2026-05-16T10:00:00-04:00
created_by: user
scope:
  type: workstream
  id: WS-STEAM-RELEASE
backend:
  kind: sqlite
  schema_version: 1
export:
  public_allowed: false
  private_control_repo_allowed: true
provenance:
  source: chatgpt
  import_method: manual
```

Low-level implementation can vary, but this is the conceptual contract.

6.4 Storage API operations

The conceptual Storage API should support:

```
put_object(object, policy)
get_object(ref)
list_objects(prefix, filters)
append_event(stream_ref, event, policy)
promote(ref, target_policy)
export(ref, target)
import_object(source)
redact(ref, strategy)
seal(ref)
delete_or_expire(ref)
verify(ref)
backup(target)
```

Important semantics:

- **promote** is explicit.
 - **redact** does not pretend erased raw data was never seen; it creates a redaction event unless the backend supports secure deletion and policy permits it.
 - **seal** makes an object immutable except for later references.
 - **verify** checks path/metadata/backend consistency.
-

6.5 Backend capability model

Each backend advertises capabilities:

YAML

```
supports_transactions: true
supports_versioning: true
supports_private_storage: true
supports_public_export: false
supports_retention: partial
supports_encryption: false
supports_concurrent_writers: single_writer
supports_search: true
supports_backup: true
```

This prevents false guarantees.

Backends:

Backend	Best use
in-memory	tests and ephemeral sessions
filesystem	simple import/export, debugability
SQLite	local private durable ledger
Git repo	curated versioned control artifacts
private control repo	multi-repo shared control plane
future service	teams, auth, sync, backups

SQLite is the recommended local ledger backend because it is self-contained and transactional. Python's `sqlite3` docs describe SQLite as a lightweight disk-based database that does not require a separate

[Python](#)

server process. [documentation](#)

7. LRH Conversation Model

7.1 Core objects

Conversation
 ConversationTurn
 ConversationEvent
 Attachment
 ContextPacket
 ModelRequest
 ModelResponse
 ToolInvocation
 PromptDraft
 RunProposal
 PromotionRecord
 EvidenceCandidate
 ConversationSummary
 ExternalReference

7.2 Event stream + artifact graph

Use both:

event stream: what happened
artifact graph: what it means

Example event stream:

```
conversation.created
turn.added
attachment.added
summary.created
prompt_draft.created
run_proposal.created
run.linked
evidence_candidate.created
promotion.accepted
conversation.sealed
```

Example artifact graph:

```
conversation
  → summary
  → design proposal draft
  → work item
  → run proposal
  → run
  → evidence
  → status interpretation
```

The event stream supports audit. The graph supports navigation and workflow.

7.3 Conversation scopes

Conversations must be scoped.

```
global
workspace
project
repository
workstream
work_item
run
evidence
release/product
```

For the Replication Vector example:

YAML

```
scope:
  type: product_release
  id: RELEASE-REPLICATION-VECTOR-STEAM
  control_repo: replication-vector-control
  controlled_repos:
    - velumin
    - replication-vector-engine
    - replication-vector-assets
    - steam-publishing
```

This lets LRH coordinate multiple repos while preserving privacy boundaries.

7.4 Conversation operations

Conceptual Conversation API:

```
create_conversation(scope, policy)
append_turn(conversation, role, content, metadata)
attach(conversation_or_turn, artifact)
build_context_packet(scope)
ask(conversation, model_provider, message)
summarize(conversation)
create_prompt_draft(conversation)
create_design_proposal_draft(conversation)
create_work_item_draft(conversation)
create_run_proposal(conversation)
promote(ref, target_kind)
import_external(source)
export(ref, target)
seal(conversation)
redact(ref, strategy)
```

Roles:

```
user
assistant
agent
tool
system
reviewer
imported_external
```

Turn content should support:

```
markdown
plain text
structured JSON
references/attachments
```

8. Promotion model

The promotion path is the heart of the design.

```
conversation
  → summary
  → draft
  → proposal
  → approved artifact
  → evidence/status
```

Examples:

```
conversation → design proposal draft → project/design/proposals/proposed/...
conversation → work item draft → project/work_items/proposed/...
conversation → prompt draft → prompt package
conversation → run proposal → execution record
conversation → evidence candidate → project/evidence/...
```

Promotion rules:

1. Promotion is explicit.
2. Promotion preserves provenance.
3. Promotion changes authority class.
4. Promotion may require redaction.
5. Promotion may target public repo, private repo, or private state.
6. Promotion should run validation where applicable.

This matches LRH's prompt workflow: meaningful prompt-driven changes use prompt IDs and execution records to make work traceable, searchable, and reviewable.

PROMPTS

9. External adapter surfaces

9.1 CLI adapter

First practical adapter.

Examples:

Bash

```
lrh conversation create --scope work-item --id WI-123
lrh conversation import transcript.md --scope workstream --id WS-ABC
lrh conversation summarize CHAT-123
lrh conversation promote CHAT-123 --to design-proposal
lrh context get --scope work-item --id WI-123 --format json
lrh run propose --from-conversation CHAT-123
```

Pros: easy to test, scriptable, vendor-neutral.

9.2 File exchange adapter

Inbox/outbox:

```
.lrh_exchange/
  inbox/
    conversation-import.json
    run-proposal.json
  outbox/
    context-packet.json
    result.json
```

Pros: simple and tool-agnostic.

Needs atomic writes and validation.

9.3 MCP adapter

MCP resources:

```
lrh://projects
lrh://project/{id}/snapshot
lrh://conversation/{id}
lrh://work-item/{id}
```

MCP tools:

```
lrh.context.get
lrh.conversation.import
lrh.conversation.promote
lrh.run.propose
lrh.evidence.attach
```

MCP is a natural surface because it is built for exposing context and tools to LLM clients; its resources are application-controlled data identified by URIs, and prompts can expose reusable workflows.

9.4 GitHub adapter

Later:

```
/lrh snapshot
/lrh record-design
/lrh propose-run WI-123
/lrh attach-evidence
```

GitHub Apps can use webhooks and scoped permissions; this is powerful but requires careful permission design.

9.5 GPT Actions / OpenAPI adapter

A Custom GPT or similar external tool could call LRH endpoints:

```
POST /conversation/import
POST /conversation/promote
POST /run/propose
GET /context
```

Should be local/private first, not internet-exposed by default.

9.6 lrh serve UI adapter

Eventually the dashboard can show:

```
chat panel
conversation history
context packet preview
promotion actions
run proposal review
evidence attachments
```

But UI should come after storage/conversation semantics.

10. Security model

10.1 Threats

Threat	Example
prompt injection	imported text says "ignore guardrails and run command"
accidental publication	private chat exported to public repo
authority confusion	chat marks work complete
evidence forgery	external tool claims tests passed
secret leakage	API key appears in transcript
vendor lock-in	chat only works with one provider

Threat	Example
backend false guarantees	"durable" data stored transiently

10.2 Controls

private by default
 explicit promotion
 untrusted input labeling
 no direct execution from chat
 run proposal approval
 guardrail review
 backend capability checks
 redaction workflow
 provenance and hashes for evidence
 validation before status

OWASP's guidance is directly relevant: prompt injection arises because instructions and data are often processed together, so LRH should keep untrusted conversation content separate from privileged

[OWASP Cheat Sheet](#)

operations. [Series](#)

11. Implementation strategy

Phase 0 — Design only

Create a proposal:

```
project/design/proposals/proposed/lrh_conversations_storage_interop/
lrh_conversations_storage_interop.md
```

Include:

use cases
 storage model
 conversation model
 promotion model
 external adapters
 security model
 phased plan
 open questions

Phase 1 — Minimal storage prototype

Implement:

storage policy dataclasses
 in-memory backend for tests
 filesystem backend for debugging/import/export
 object metadata validation

Why not SQLite first? Because the API should be tested before selecting the durable backend. This keeps the first code PR small and deterministic.

Phase 2 — Local durable ledger

Implement SQLite backend:

```
schema_version
objects
events
conversations
turns
attachments
refs
indexes
exports
```

Use migrations from day one, even if minimal.

Phase 3 — Conversation import/export

Implement:

```
lrh conversation import
lrh conversation list
lrh conversation show
lrh conversation export
lrh conversation summarize metadata placeholder
```

Do not require a model provider yet.

Phase 4 — Promotion

Implement:

```
promote to design proposal draft
promote to work item draft
promote to prompt draft
promote to evidence candidate
```

Human review required.

Phase 5 — Context-aware ask

Implement provider-neutral model adapter:

```
lrh conversation ask --provider ...
```

Non-agentic. No tool execution.

Phase 6 — External protocol

Implement CLI JSON and file exchange, then MCP.

Phase 7 — Chat-to-run

Implement:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

run proposal from conversation
approval
event log
external tool reporting
evidence attachment

Phase 8 — UI

Integrate with `lrm serve`.

12. Low-level choices still needed

Storage

- SQLite file location.
- Filesystem layout for private state.
- Whether to support encryption at rest initially.
- Backup/export format.
- Redaction semantics.
- Retention enforcement mechanism.
- Schema migration approach.
- Single-writer vs multi-writer assumptions.

Conversation

- Exact JSON schema.
- Whether turns are immutable.
- Attachment storage location.
- How to hash attachments.
- How to store provider metadata.
- How to represent imported vendor transcripts.
- How to represent summaries and model-generated text.

Interop

- CLI names: **chat** vs **conversation**.
- JSON schema versioning.
- MCP resource URI design.
- GitHub slash command vocabulary.

- How authentication works for local APIs.
- Whether external tools can request context without user confirmation.

Policy

- Public export review requirements.
 - Secret detection/redaction level.
 - Run approval requirements.
 - Distinction between evidence candidate and accepted evidence.
 - Storage fallback behavior when backend guarantee is insufficient.
-

13. Pros and cons of the unified design

Pros

Strong user-control story

The user keeps their development process data under their own control.

Future-tool friendly

LRH does not need to beat ChatGPT, Codex, Claude, Aider, Kiro, or future tools. It gives them a protocol.

Aligns with LRH philosophy

It preserves explicit state, evidence, validation, and scoped execution.

Avoids premature chatbot build

The design prioritizes storage, promotion, and interop before fancy UI.

Supports multi-repo products

The private control repo pattern becomes first-class.

Enables gradual adoption

Manual import/export can work before MCP or dashboard chat exists.

Cons

More abstract upfront

A storage/conversation API is more design work than a single chat import command.

Risk of over-engineering

Capability matrices, URI schemes, and policy classes can become too elaborate.

Mitigation: design them now, implement only a minimal subset first.

Storage security is hard

Privacy bugs are serious.

Mitigation: private by default, no public raw export, explicit promotion.

Conversation data can become huge

Raw transcripts, attachments, and model outputs grow quickly.

Mitigation: retention classes, summaries, archives, export controls.

External protocols are moving targets

MCP, vendor APIs, and GitHub agent surfaces will evolve.

Mitigation: keep LRH's internal conversation and storage model independent of adapter protocols.

14. Best-practice evaluation

Against LRH architecture

Good. It preserves the separation between control artifacts and runtime structured objects. LRH already requires clear boundaries between harness code, project control planes, scripts, and future client repositories.

AGENTS

Against durability/storage practice

Good if implemented with explicit backend guarantees. SQLite is a reasonable local durable ledger because it is embedded and transactional, but LRH must not overpromise beyond backend settings and filesystem realities. [SQLite](#)

Against version-control practice

Good if Git is used for curated artifacts, not raw chat logs. Git is a version-control system for recording file changes over time; it is not a general-purpose high-frequency private event database. [Git](#)

Against AI security practice

Good if untrusted chat content is quarantined from privileged actions. OWASP's prompt injection guidance supports this separation because LLMs often process natural-language instructions and data

[OWASP Cheat Sheet](#)

together. [Series](#)

Against LRH PR discipline

Good if staged. LRH's style guide favors small, scoped, reviewable, deterministic changes and warns against broad speculative rewrites.

STYLE

15. Recommended design proposal outline

LRH Conversations, Storage, and External Agent Interop

Summary

Motivation

Use Cases

- @lrh record/promote this
- dashboard contextual chat
- external tool protocol
- private durable ledger
- multi-repo control plane
- chat-to-run cockpit

Non-Goals

- custom LRH-trained LLM
- full chatbot clone
- raw public transcript storage by default
- direct unapproved execution from chat
- vendor-specific implementation as the core

Design Principles

- conversations are non-authoritative by default
- private by default
- explicit durability
- external tools propose; LRH decides
- Git for curated artifacts, not raw chat database

LRH Storage Model

- conceptual URI namespace
- privacy classes
- durability classes
- retention classes
- authority classes
- object metadata
- storage API operations
- backend capability model
- private control repo pattern

LRH Conversation Model

- core objects
- event stream
- artifact graph
- scopes
- operations
- summaries
- attachments
- provider metadata

Promotion Model

- conversation → summary → draft → proposal → artifact
- promotion rules
- validation hooks

External Adapter Surfaces

- CLI
- file exchange
- MCP
- GitHub
- GPT Actions/OpenAPI
- lrh serve UI

Security and Privacy

- threat model
- untrusted content boundaries
- approval model
- redaction
- provenance
- secret handling

Phased Implementation

- design
- storage prototype
- durable ledger
- import/export
- promotion
- context-aware ask
- external protocol
- chat-to-run
- UI

Open Questions

References

16. Final recommendation

Greenlight the design proposal.

Do not greenlight full implementation yet. The next correct step is a design PR that records the unified architecture and explicitly prevents premature implementation.

The design should commit to these core decisions:

1. LRH Conversations are non-authoritative until promoted.
2. Raw conversations are private by default.
3. Storage has explicit privacy, durability, retention, and authority policies.
4. Git stores curated control artifacts, not raw chat by default.
5. External tools integrate through stable protocol surfaces.
6. Chat-to-run requires explicit proposal, approval, evidence, and status interpretation.

That gives LRH a credible path toward something genuinely valuable: an open, user-controlled, evidence-backed AI development control plane that can work with any good agent tool without surrendering the developer's data, process, or audit trail to a silo.

Provide a structured prompt to generate a thorough design PR to accomplish this task: record the unified architecture to guide implementation in a proper order. The design should commit to these core decisions:

1. LRH Conversations are non-authoritative until promoted.
2. Raw conversations are private by default.
3. Storage has explicit privacy, durability, retention, and authority policies.
4. Git stores curated control artifacts, not raw chat by default.
5. External tools integrate through stable protocol surfaces.
6. Chat-to-run requires explicit proposal, approval, evidence, and status interpretation.

That gives LRH a credible path toward something genuinely valuable: an open, user-controlled, evidence-backed AI development control plane that can work with any good agent tool without

surrendering the developer's data, process, or audit trail to a silo.

Before writing the prompt, generate a prompt ID using the conventions in **PROMPTS.md**. Include that prompt ID at the top of the generated prompt.

After the PR is generated, the agent should create an execution record for the prompt using **scripts/prompts/record-execution**, following the conventions in **PROMPTS.md** and **project/executions/README.md**.

In that prompt, ask the agent to:

- follow AGENTS.md
- follow STYLE.md for consistency
- follow PROMPTS.md for prompt IDs, execution records, and rerun handling
- update any README.md's in affected folders
- add or update an execution record for the prompt (using AD_HOC if no work item applies)
- check for prior execution records for the same prompt ID and apply soft idempotence rules before proceeding

Keep the workflow lightweight: execution records are expected for meaningful prompt-driven PRs, but should not block small or obvious changes. Do not update execution records unrelated to this task.

Target the prompt to Codex Cloud and provide it as both a downloadable file AND a copy-and-paste block inline within the chat. Please present the downloadable file first, as prompts in copy-and-paste blocks sometimes interfere with the rendering of the chat. The copy-and-paste block should be a 1-1 copy of the prompt; NEVER present a copy and paste block that does not exactly correspond to the exact contents of the downloadable file.

I'll create a single Codex Cloud-ready prompt file for the design PR, then mirror it exactly inline.

Markdown

PROMPT(AD_HOC:LRH_CONVERSATIONS_STORAGE_INTEROP_DESIGN)[2026-05-16T00:30:00-04:00]

Target: Codex Cloud

Repository: xenotaur/logical_robotics_harness

Task: Create a thorough design PR that records the unified architecture for LRH Conversations, Storage, and External Agent Interop, guiding future implementation in the proper order.

Context:

LRH is evolving toward an open, user-controlled, evidence-backed AI development control plane. A significant amount of real AI-assisted development happens today in external chat and agent tools such as ChatGPT, Codex, Claude Code, GitHub Copilot, Aider, Kiro-like tools, and future tools. LRH should not try to replace every tool or train a bespoke LRH LLM as the first move. Instead, LRH should define a durable, private-by-default conversation/storage model and stable protocol surfaces so conversations and external agent work can be captured, scoped, promoted, audited, and integrated with LRH artifacts.

This PR is design-only. It should record the unified architecture and staged plan, not implement the storage layer, chat UI, MCP server, GitHub App, model provider integration, or run execution behavior.

Core design commitments:

Printed using [ChatGPT to PDE](#), powered by PDFCrowd [HTML to PDF API](#).

1. LRH Conversations are non-authoritative until promoted.
2. Raw conversations are private by default.
3. Storage has explicit privacy, durability, retention, and authority policies.
4. Git stores curated control artifacts, not raw chat by default.
5. External tools integrate through stable protocol surfaces.
6. Chat-to-run requires explicit proposal, approval, evidence, and status interpretation.

Required process:

1. Follow `AGENTS.md`.
2. Follow `STYLE.md` for consistency and review discipline.
3. Follow `PROMPTS.md` for prompt IDs, execution records, and rerun handling.
4. Check for prior execution records for this exact prompt ID before starting:
`PROMPT(AD\_HOC:LRH\_CONVERSATIONS\_STORAGE\_INTEROP\_DESIGN)[2026-05-16T00:30:00-04:00]`
5. Apply the soft idempotence rules from `PROMPTS.md`:
 - If a prior `landed` or `in\_progress` execution exists for this exact prompt ID, stop and report it.
 - If a prior `failed`, `reverted`, or `superseded` execution exists, summarize it and proceed only if this is an intentional rerun/follow-up.
 - If status is ambiguous, stop and report the ambiguity.
6. Keep the workflow lightweight. Execution records are expected for this meaningful design PR, but do not update execution records unrelated to this task.

Design placement:

Create a proposed design document under the repository's current design proposal conventions. Prefer a dedicated proposed design directory if that is consistent with nearby proposal layout, for example:

```
```text
project/design/proposals/proposed/lrh_conversations_storage_interop/
 lrh_conversations_storage_interop.md
```

If the repository uses a different convention for proposed design documents, follow the existing convention and preserve the same design content.

Use frontmatter consistent with existing design proposal metadata. If the repository convention permits, use an ID similar to:

#### YAML

```
id: DESIGN-LRH-CONVERSATIONS-STORAGE-INTEROP
status: proposed
```

#### Design content requirements:

The design document should be thorough and should include at least the following sections.

##### 1. Summary

Explain that the proposal defines LRH Conversations, Storage, and External Agent Interop as a unified architecture for making AI-assisted development conversations and tool work auditable, private-by-default, scoped, promotable, and interoperable.

State explicitly that LRH should become a control-plane hub and protocol layer around many tools, not a replacement for every chat or agent tool.

##### 2. Motivation

Cover:

- AI-assisted development work often happens in opaque external chats and agent tools.
- Useful reasoning, decisions, prompts, failures, and handoffs can remain trapped in vendor silos.
- LRH's value is explicit, evidence-backed, human-auditable, machine-interpretable workflow state.

- The goal is to preserve user control over data and process while allowing use of best-of-breed tools.
- LRH should support future tools without requiring users to abandon current tools.

### 3. Target use cases

Include and describe these use cases:

- **@lrh record this / @lrh promote this** from an external chat.
  - Example outcomes: design proposal draft, work item draft, prompt package, evidence candidate, run proposal.
- Contextual chat from the LRH meta/project dashboard.
  - Example: select a project in “Needs Attention,” chat about what to do next, draft a work item, or propose a run.
- External tool integration API.
  - Example: a GitHub, OpenAI, Anthropic, Aider, Kiro-like, or future tool asks LRH for context, proposes work, reports progress, and attaches evidence.
- Durable private conversation ledger.
  - Raw conversations are stored privately, are searchable/exportable, and can be curated into repo artifacts.
- Private multi-repo control plane.
  - Example: one private control repo coordinates public code repos, private asset repos, and private publishing/release repos.
- Chat-to-run cockpit.
  - Conversation produces prompt draft, run proposal, approval, execution events, evidence, and derived status.

For each use case, summarize:

- what it enables;
- why it fits LRH;
- major risks;
- minimum technical requirements;
- existing alternatives and why LRH is still useful.

### 4. Non-goals

Include explicit non-goals:

- Do not build a full LRH-native chatbot as the first implementation.
- Do not train or require an LRH-specific LLM.
- Do not store raw conversations in public repos by default.
- Do not use Git as the live database for raw chat/event streams by default.
- Do not allow chats or external agents to directly mark work complete or mutate authoritative status.

- Do not build deep MCP, GitHub App, or vendor-specific integrations before the storage and conversation semantics are defined.
- Do not implement this proposal in this PR.

## 5. Core decisions

Record these as explicit design decisions:

- LRH Conversations are non-authoritative until promoted.
- Raw conversations are private by default.
- Storage has explicit privacy, durability, retention, and authority policies.
- Git stores curated control artifacts, not raw chat by default.
- External tools integrate through stable protocol surfaces.
- Chat-to-run requires explicit proposal, approval, evidence, and status interpretation.

Explain the consequence of each decision.

## 6. High-level architecture

Describe the layered architecture:

**LRH authoritative control artifacts**

**design / work\_items / evidence / status / executions**

**LRH Conversation Model**

**conversations / turns / events / summaries / proposals**

**LRH Storage API**

**privacy / durability / retention / versioning / authority**

**Storage backends**

**in-memory / filesystem / SQLite / Git exports / private control repo / future service**

**External adapters**

**CLI / file exchange / MCP / GitHub / GPT Actions/OpenAPI / lrh serve UI**

Emphasize that adapters sit above stable conceptual APIs.

## 7. LRH Storage Model

Define a conceptual storage model, including:

- LRH URI/address namespace, for example:
  - **lrh://transient/private/conversations/local/CHAT-123**
  - **lrh://versioned/private/conversations/WS-STEAM-RELEASE/CHAT-456**
  - **lrh://versioned/public/evidence/WI-123/EV-789**
  - **lrh://sealed/shared\_private/runs/RUN-456**
- Privacy classes:
  - **private**
  - **shared\_private**
  - **public**
  - **secret**
- Durability classes:

- **transient**
- **session**
- **durable**
- **versioned**
- **sealed**
- Retention classes:
  - **ttl=<duration>**
  - **keep**
  - **archive\_after=<duration>**
  - **delete\_after=<duration>**
  - **manual**
- Authority classes:
  - **non\_authoritative\_context**
  - **draft**
  - **proposal**
  - **approved\_run**
  - **evidence\_candidate**
  - **accepted\_evidence**
  - **authoritative\_control\_artifact**
- Object metadata, including:
  - ID
  - kind
  - URI
  - privacy
  - durability
  - retention
  - authority
  - timestamps
  - scope
  - backend
  - export policy
  - provenance
- Path/URI policy and metadata policy must agree.
- Moving an object must not silently change privacy or authority.
- Promotion/export must be explicit.

## 8. Storage API operations

Define conceptual operations, not necessarily code:

- **put\_object**
- **get\_object**
- **list\_objects**
- **append\_event**
- **promote**
- **export**
- **import\_object**
- **redact**
- **seal**
- **delete\_or\_expire**
- **verify**
- **backup**

Clarify expected semantics:

- **promote** changes authority and/or exportability only through explicit reviewable action.
- **redact** should be modeled as a redaction event unless secure deletion is explicitly supported and policy permits it.
- **seal** makes an object immutable except for later references.
- **verify** checks path/metadata/backend consistency.

## 9. Backend capability model

Document that storage backends must advertise capabilities, such as:

- transactions
- versioning
- private storage
- public export
- retention enforcement
- encryption support
- concurrent writers
- search
- backup

Discuss likely backend roles:

- in-memory for tests and ephemeral sessions;
- filesystem for simple import/export and debugging;
- SQLite for local private durable ledger;



- Git for curated control artifacts;
- private control repo for multi-repo shared control plane;
- future service for team/server workflows.

Be explicit that Git is not the default raw chat database.

## 10. LRH Conversation Model

Define core objects:

- Conversation
- ConversationTurn
- ConversationEvent
- Attachment
- ContextPacket
- ModelRequest
- ModelResponse
- ToolInvocation
- PromptDraft
- RunProposal
- PromotionRecord
- EvidenceCandidate
- ConversationSummary
- ExternalReference

Define event-stream plus artifact-graph structure:

- event stream records what happened;
- artifact graph records what the conversation produced or linked to.

Example events:

- **conversation.created**
- **turn.added**
- **attachment.added**
- **summary.created**
- **prompt\_draft.created**
- **run\_proposal.created**
- **run.linked**
- **evidence\_candidate.created**
- **promotion.accepted**
- **conversation.sealed**

## 11. Conversation scopes

Define possible scopes:

- global
- workspace
- project
- repository
- workstream
- work item
- run
- evidence
- release/product

Include the multi-repo Replication Vector style example:

- public graphics library repo;
- public/private game engine repo;
- private assets/levels repo;
- private Steam publishing repo;
- one private LRH control repo coordinating them.

## 12. Conversation API operations

Define conceptual operations:

- **create\_conversation**
- **append\_turn**
- **attach**
- **build\_context\_packet**
- **ask**
- **summarize**
- **create\_prompt\_draft**
- **create\_design\_proposal\_draft**
- **create\_work\_item\_draft**
- **create\_run\_proposal**
- **promote**
- **import\_external**
- **export**
- **seal**

- **redact**

Clarify that **ask** is non-agentic by default and should not execute tools or mutate project state unless a later explicit run/action design authorizes it.

### 13. Promotion model

Define promotion path:

```
conversation
-> summary
-> draft
-> proposal
-> approved artifact
-> evidence/status
```

Examples:

- conversation -> design proposal draft -> **project/design/proposals/proposed/...**
- conversation -> work item draft -> **project/work\_items/proposed/...**
- conversation -> prompt draft -> prompt package
- conversation -> run proposal -> execution/run record
- conversation -> evidence candidate -> **project/evidence/...**

Promotion rules:

- explicit;
- provenance-preserving;
- may require redaction;
- may target private state, private control repo, or public repo;
- should run validation where applicable;
- does not allow chat to bypass LRH authority and evidence rules.

### 14. External adapter surfaces

Describe intended adapter surfaces without implementing them:

- CLI adapter:
  - **lrh conversation create**
  - **lrh conversation import**
  - **lrh conversation summarize**
  - **lrh conversation promote**
  - **lrh context get --format json**
  - **lrh run propose --from-conversation ...**
- File exchange adapter:
  - inbox/outbox JSON files for tools that cannot call a server;
  - atomic write and schema validation requirements.

- MCP adapter:
  - resources such as projects, snapshots, conversations, work items;
  - tools such as context get, conversation import/promote, run propose, evidence attach.
- GitHub adapter:
  - future slash commands or PR comments;
  - evidence links and run progress.
- GPT Actions/OpenAPI or similar adapter:
  - local/private endpoint for external chat tools to call into LRH;
  - not internet-exposed by default.
- **lhr serve** UI adapter:
  - eventual dashboard chat panel, context preview, promotion actions, run proposal review.

Emphasize that these are adapters over the stable conceptual APIs, not the foundation.

## 15. Security and privacy model

Include a threat model and controls.

Threats:

- prompt injection;
- accidental publication;
- authority confusion;
- evidence forgery;
- secret leakage;
- vendor lock-in;
- backend false guarantees;
- unsafe tool execution.

Controls:

- private by default;
- explicit promotion/export;
- untrusted input labeling;
- no direct execution from chat;
- run proposal and guardrail review;
- backend capability checks;
- redaction workflow;
- provenance and source links;
- validation before status interpretation;

- do not expose local APIs publicly by default.

## 16. Implementation phases

Define implementation phases in proper order:

- Phase 0: design only.
- Phase 1: minimal storage prototype:
  - policy dataclasses or typed models;
  - in-memory backend for tests;
  - filesystem backend for debugging/import/export;
  - metadata validation.
- Phase 2: local durable ledger:
  - SQLite backend;
  - schema versioning;
  - conversations, turns, events, attachments, refs, indexes, exports.
- Phase 3: conversation import/export:
  - `lrm conversation import/list/show/export`;
  - no model provider required yet.
- Phase 4: promotion:
  - promote to design proposal draft, work item draft, prompt draft, evidence candidate.
- Phase 5: context-aware ask:
  - provider-neutral non-agentic model adapter;
  - no tool execution.
- Phase 6: external protocol:
  - CLI JSON and file exchange first;
  - MCP later.
- Phase 7: chat-to-run:
  - run proposal from conversation;
  - approval;
  - event log;
  - external tool reporting;
  - evidence attachment.
- Phase 8: UI:
  - integrate with `lrm serve`.

## 17. Low-level choices still open

Record open questions, including:

- exact storage paths;
- SQLite database location;
- encryption-at-rest posture;
- schema migration approach;
- retention enforcement;
- redaction semantics;
- single-writer vs multi-writer assumptions;
- attachment hashing;
- provider metadata schema;
- CLI naming: **chat** vs **conversation**;
- MCP URI design;
- GitHub slash command vocabulary;
- local API authentication;
- whether external tools can request context without user confirmation.

## 18. Pros, cons, and tradeoffs

Include a section evaluating the design:

Pros:

- user control over data;
- future-tool friendly;
- avoids vendor lock-in;
- aligns with LRH evidence/status philosophy;
- supports multi-repo control planes;
- enables gradual adoption.

Cons:

- abstract upfront;
- storage security is hard;
- conversation data can grow large;
- external protocols evolve;
- risk of over-engineering.

Mitigations:

- design first;
- minimal implementation phases;
- private by default;

- explicit promotion;
- do not implement deep integrations until storage/conversation semantics are stable.

## 19. Relationship to existing LRH docs

Link or reference relevant LRH docs where appropriate, such as:

- architecture/design docs;
- meta workspace/control-plane specs;
- `lrh serve` visual language proposal if relevant;
- prompt workflow documentation;
- work-item/execution docs if relevant.

Documentation updates:

- Update any README.md files in affected folders, if present and appropriate.
- If `project/design/proposals/proposed/README.md` or a design proposal index exists, add this proposal there.
- If `project/design/README.md` or a design index exists and conventionally lists proposals, add a concise entry there.
- If there is a roadmap/focus/workstream document for future agent integration, chat, storage, or execution framework, add a concise non-blocking reference only if clearly appropriate.
- Do not make broad unrelated documentation edits.
- Do not update execution records unrelated to this task.

Validation:

Run the repository's relevant validation/documentation checks. At minimum, try:

**Bash**

`lrh validate`

If available and appropriate, also run:

**Bash**

`scripts/test`  
`scripts/lint`  
`scripts/format --check`

If commands are unavailable in Codex Cloud, report that clearly and run the closest supported alternative. Inspect `git diff` to ensure the PR is scoped to the design proposal, relevant README/index references, and the execution record.

Execution record:

After making the changes, create an execution record using `scripts/prompts/record-execution`, following `PROMPTS.md` and `project/executions/README.md` conventions. Use `AD_HOC` if no work item applies.

Use this exact prompt ID:

`PROMPT(AD_HOC:LRH_CONVERSATIONS_STORAGE_INTEROP_DESIGN)[2026-05-16T00:30:00-04:00]`

The execution record should summarize:

- the design proposal created;
- the core decisions recorded;
- README/index references updated;
- validation commands run and results;
- any follow-up work identified.

Review constraints:

- Keep the PR design/documentation-only.
- Do not implement storage code, conversation code, chat UI, MCP, GitHub, HTTP, model provider, or run execution behavior.
- Do not choose a final backend implementation beyond documenting the recommended phased approach.
- Do not alter unrelated design proposals, execution records, work items, or project-control artifacts.
- Keep diffs explicit, scoped, and reviewable.

:contentReference[oaicite:0]{index=0}

This design is in at [https://github.com/xenotaur/logical\\_robotics\\_harness/blob/main/project/design/proposals/proposed/lrh-conversations-storage-interop/00\\_proposal.md](https://github.com/xenotaur/logical_robotics_harness/blob/main/project/design/proposals/proposed/lrh-conversations-storage-interop/00_proposal.md) . Does it capture everything it needs to?

Yes. It captures the architecture we discussed well enough to serve as the design proposal and guide future implementation.

The important parts are all present:

Concern	Captured?	Evidence
Design-only scope	Yes	The proposal says it records storage semantics, conversation semantics, protocol boundaries, safety rules, and implementation phasing, and explicitly does not implement storage, chat UI, MCP, GitHub App, provider integration, HTTP, or run execution. <a href="#">GitHub</a>
Core six decisions	Yes	The proposal records all six: non-authoritative conversations, private raw conversations, explicit storage policies, Git for curated artifacts, stable protocol surfaces, and chat-to-run requiring proposal/approval/evidence/status interpretation. <a href="#">GitHub</a>
Target use cases	Yes	It covers <b>@lrh record/promote</b> , dashboard contextual chat, external tool API, durable private ledger, private multi-repo control plane, and chat-to-run cockpit. <a href="#">GitHub</a>
Storage model	Yes	It defines LRH URI examples, privacy classes, durability classes, retention classes, authority classes, and object metadata. <a href="#">GitHub</a>
Storage API	Yes	It defines conceptual operations including <b>put_object</b> , <b>append_event</b> , <b>promote</b> , <b>export</b> , <b>redact</b> , <b>seal</b> , <b>verify</b> , and <b>backup</b> , with expected semantics. <a href="#">GitHub</a>



Concern	Captured?	Evidence
Backend capability model	Yes	It requires backends to advertise capabilities and gives roles for in-memory, filesystem, SQLite, Git, private control repo, and future service backends. <a href="#">GitHub</a>
Conversation model	Yes	It defines the core objects and the event-stream plus artifact-graph structure. <a href="#">GitHub</a>
Conversation scopes and multi-repo example	Yes	It includes global/workspace/project/repository/workstream/work item/run/evidence/release scopes and the Replication Vector-style multi-repo example. <a href="#">GitHub</a>
Promotion model	Yes	It gives the conversation → summary → draft → proposal → approved artifact → evidence/status path and explicitly states promotion rules. <a href="#">GitHub</a>
External adapter surfaces	Yes	It covers CLI, file exchange, MCP, GitHub, GPT Actions/OpenAPI, and <b>l rh serve</b> UI adapters. <a href="#">GitHub</a>
Security model	Yes	It lists threats and controls, including prompt injection, accidental publication, authority confusion, evidence forgery, secret leakage, backend false guarantees, private-by-default storage, explicit promotion/export, and no direct execution from chat. <a href="#">GitHub</a>
Implementation order	Yes	It defines phases from design-only through minimal storage, SQLite ledger, import/export, promotion, context-aware ask, external protocol, chat-to-run, and UI. <a href="#">GitHub</a>

## My review

I would call this **complete enough to keep**. I do not see a blocking omission.

The design does what it needed to do: it records the architecture, locks in the key safety/privacy decisions, avoids premature implementation, and creates an implementation sequence that starts with storage/conversation semantics before adapters, chat UI, or agentic runs.

## Minor follow-up polish worth considering

I would not block on these, but I would consider a small follow-up later:

### 1. Add explicit schema/protocol versioning language.

The proposal mentions schema versioning in the SQLite phase, but external adapter schemas should probably also have explicit `schema_version` / `protocol_version` expectations.

### 2. Add a tiny “first implementation acceptance criteria” section.

For Phase 1, something like: typed policy models, in-memory backend, metadata validation, tests for privacy/durability/authority mismatch, and no chat/model integration yet.

### 3. Add cost/accounting as an open question.

For future **ask** and model-provider use, token cost, provider attribution, and budget guardrails will matter.

### 4. Add secret-detection posture as an open question.

The proposal includes redaction and secret leakage, but it might later need to distinguish manual redaction, best-effort scanning, and stronger secret-detection integrations.

None of those undermines the proposal. They are natural follow-ons.

## Recommended next step

Treat this as a solid Phase 0 design artifact. The next implementation-planning step should be a prompt package for the earliest slices:

PR1: Storage policy/types and in-memory backend  
 PR2: Filesystem import/export backend and metadata validation  
 PR3: Conversation core model and event stream tests  
 PR4: CLI import/list/show/export

I would not jump directly to SQLite, MCP, or dashboard chat until the policy model and conversation object model are proven in small tests.

Let's look at these PRs from a user feature lens. What command or pattern would they enable that we could dogfood at the end of this?

At the end of those first implementation PRs, the dogfoodable user feature should be:

**"I had an important external AI conversation; LRH can now capture it privately, scope it to a project/work item, preserve it with explicit storage policy metadata, list/show it later, and export it in a reviewable format."**

That is not yet "@lrh run this" or "chat inside the dashboard." It is the first useful brick: **conversation capture under user control**.

## The user-facing pattern

A plausible first dogfood flow:

```
Bash
1. Capture a useful external chat transcript into private LRH state.
lrh conversation import chatgpt-lrh-storage-design.md \
 --scope work-item WI-LRH-SERVE-SAFE-DEFAULT-MVP \
 --privacy private \
 --durability versioned \
 --source chatgpt

2. See what conversations are attached to that work item.
lrh conversation list \
 --scope work-item WI-LRH-SERVE-SAFE-DEFAULT-MVP

3. Inspect the captured conversation.
lrh conversation show CHAT-20260516-001

4. Export it for review, backup, or later curation.
lrh conversation export CHAT-20260516-001 \
 --format markdown \
 --out /tmp/chatgpt-lrh-storage-design.export.md
```

The immediate dogfood story is:

External chat → LRH private conversation record → scoped retrieval → exportable artifact

That is already useful because it addresses the hidden-chat problem without attempting full agentic execution.

## What each PR enables from a feature lens

PR	Engineering slice	User-facing value
PR1	Storage policy/types + in-memory backend	LRH can represent “private, versioned, non-authoritative conversation state” correctly and testably.
PR2	Filesystem import/export backend + metadata validation	User can store and retrieve conversation-like artifacts outside the repo with policy metadata.
PR3	Conversation core model + event stream	LRH can distinguish conversations, turns, events, attachments, provenance, and scopes.
PR4	CLI import/list/show/export	User can actually capture an external chat and find it later.

So the first dogfood command family is:

### Bash

```
lrh conversation import
lrh conversation list
lrh conversation show
lrh conversation export
```

I would probably also include one validation/debug command if cheap:

### Bash

```
lrh conversation verify CHAT-20260516-001
```

or:

### Bash

```
lrh storage verify lrh://versioned/private/conversations/...
```

That would make the storage-policy work visible to the user.

## What this does not enable yet

It does not yet enable:

### Bash

```
lrh conversation ask
lrh conversation promote
lrh run propose --from-conversation
```

Those are the next real feature layers.

The first tranche is basically “**LRH can remember this conversation safely.**”

The second tranche is “**LRH can turn this conversation into a draft artifact.**”

The third tranche is “**LRH can turn this conversation into a run proposal.**”

## The best next dogfood milestone after import/list/show/export

The next high-value feature after the first four PRs is probably:

**Bash**

```
lrh conversation promote CHAT-20260516-001 \
 --to design-proposal-draft \
 --out project/design/proposals/proposed/lrh-example/00_proposal.md
```

or more safely:

**Bash**

```
lrh conversation draft CHAT-20260516-001 \
 --kind design-proposal \
 --out /tmp/lrh-example-design-proposal.md
```

I prefer **draft** before **promote**, because it keeps the authority boundary clean. A conversation is non-authoritative until explicitly promoted, and the draft step makes that boundary visible.

## Recommended dogfood target

The first complete dogfood scenario should be:

```
Have a useful ChatGPT conversation about LRH.
Save/export/copy it to Markdown.
Run `lrh conversation import`.
Attach it to a work item or workstream.
List/show it later.
Export it.
Use the exported conversation manually to draft the next design/work item.
```

That is enough to prove the storage/conversation foundation is real, while staying consistent with LRH's current discipline: small, scoped, typed, deterministic, and not jumping straight into deep agent orchestration. LRH's own guidance warns against premature deep orchestration and emphasizes evidence-backed status and clear source/runtime boundaries.

AGENTS

Let's talk about the first two steps:

- Have a useful ChatGPT conversation about LRH.
- Save/export/copy it to Markdown.

As far as I know, the first step is possible - we're doing it now! :-D - but the second step is not. ChatGPT seems to not support saving chats as Markdown. You can get a third-party app to save to PDF via a Chrome plugin, or conduct a chat in the Codex desktop app and copy a chat to clipboard, but not from ChatGPT on the web proper. Is this feature available on the ChatGPT desktop app?

As far as I can tell from OpenAI's current help docs: **no, the ChatGPT desktop app does not appear to provide a first-class "export this conversation as Markdown" feature.**

What OpenAI officially documents today is:

1. **Account-level data export**, which sends you a ZIP containing chat history and other account data. OpenAI documents this under Settings → Data Controls → Export Data; the export link is emailed and expires after 24 hours. Chat exports are not available for ChatGPT Business or Enterprise accounts. [OpenAI Help Center](#)

2. **Mobile data export**, at least on Android, through Data Controls → Export Data, again as a downloadable ZIP sent by email/text. [OpenAI Help Center](#)
3. **Shared links**, which expose a conversation through a `chatgpt.com/share/...` URL, but OpenAI warns shared links do not have granular permissions and anyone with the link can view the conversation, so they should not be used for sensitive content. [OpenAI Help Center](#)
4. **Canvas copy/share**, but that applies to Canvas assets/documents/code, not to exporting an ordinary chat thread as Markdown. [OpenAI Help Center](#)
5. **macOS desktop Record mode**, but that is for recording/transcribing meetings, brainstorming, or voice notes, not exporting an existing text chat as Markdown. It is macOS-only and available on specified paid/workspace plans. [OpenAI Help Center](#)

So for LRH dogfooding, I would assume **ChatGPT desktop is not a reliable Markdown-export source** unless you manually copy content or use the account-level data export and convert its JSON/HTML-ish contents later.

The practical dogfood options are probably:

1. Manual select/copy from ChatGPT → save as Markdown
2. ChatGPT account data export ZIP → parse `conversations.json` later
3. Shared link → import/scrape only for non-sensitive chats
4. Ask ChatGPT to summarize the conversation into a Markdown artifact, then copy that
5. Conduct the conversation in a tool with better transcript export/copy support

For LRH's first `conversation import`, the most realistic early target may be:

**Bash**

```
lrm conversation import transcript.md --source manual --scope work-item WI-...
```

Then a later, more useful adapter could be:

**Bash**

```
lrm conversation import chatgpt-export.zip --source chatgpt-data-export
```

That second path might actually be better aligned with LRH: it treats ChatGPT as an external silo with an export format, then normalizes it into LRH's private conversation ledger.